

Consolidated Quantitative Finance and Stochastic Calculus Notebook

Consolidated Quantitative Finance and Stochastic Calculus Notebook

Integrated sources: Laboratorio 1, Laboratorio 2, Laboratorio 3, Laboratorio 4, Proyecto 4, Untitled3, Untitled5, Series de Tiempo, and Calculo Estocastico.

This consolidated notebook documents the code, formulas, assumptions, workflow, observed outputs, and financial/statistical interpretation of the source notebooks. Original code logic is preserved: code is copied as source, not refactored, optimized, reordered, or silently repaired. When a source block is fragile or execution-blocking, the issue is documented explicitly.

Executive Summary

The notebooks form a coherent quantitative finance and stochastic calculus sequence. The first group estimates historical log returns and volatility, calibrates geometric Brownian motion (GBM), simulates price paths, and prices options by Monte Carlo and Black-Scholes. A second group extends option pricing to early exercise through Longstaff-Schwartz regression for an American put and path dependence through a Parisian down-and-out barrier option. A third group applies conditional volatility and time-series modelling: GARCH(1,1), AR/ARIMA, SARIMA, Prophet, AIC, BIC, residual diagnostics, and forecasting. The final stochastic/probability notebook reviews sample spaces, Cartesian products, dice outcomes, and Bayes' theorem.

The integration is not a numerical rerun. It is a documentation and consolidation artifact. Several notebooks depend on external data or internet calls (`yfinance`, a GitHub CSV, Yahoo option chains, `prophet`, `arch`). Some outputs are therefore date-dependent. No missing result has been invented. When a value is mentioned, it comes from the saved output inside the source notebooks.

Main technical conclusions visible in the saved outputs:

- Historical volatility is used as a backward-looking calibration input for GBM and Black-Scholes, while implied volatility is inferred from option market prices and can diverge materially from historical volatility.
- Monte Carlo European option pricing converges toward Black-Scholes when both use the same risk-neutral GBM assumption.
- American puts can have a positive early-exercise premium; Longstaff-Schwartz estimates this by comparing immediate exercise against a regression estimate of continuation value.
- Parisian barrier options show the value of temporal memory: increasing the required number of consecutive days below the barrier makes knock-out less likely and increases the option

price.

- GARCH(1,1) captures volatility clustering through the ARCH shock term and the GARCH persistence term.
- AIC/BIC formalize the trade-off between fit and parsimony; BIC typically penalizes complexity more strongly.
- The time-series notebook contains real execution-order problems and one explicit `NameError`; these are flagged, not silently corrected.

Consolidated Notebook Roadmap

Order	Integrated section	Main source notebooks	Purpose
1	Log returns and GBM simulation	Laboratorio 1	Estimate daily log returns, annualize o
2	Rolling realized volatility	Laboratorio 2	Compute 21/63/126-day annualized ro
3	European option pricing	Laboratorio 3	Price a call/put by risk-neutral Monte
4	American put pricing	Laboratorio 4	Estimate an American put using Long
5	Black-Scholes with market option data	Proyecto 4	Compare Black-Scholes with historical
6	Parisian barrier options	Untitled5	Price a down-and-out Parisian option
7	GARCH volatility analysis	Untitled3	Fit GARCH(1,1), extract conditional
8	AR/ARIMA/SARIMA/Prophet	Series de Tiempo	Simulate AR(1), compare ARIMA mo
9	Probability and stochastic exercises	Calculo Estocastico	Review sample spaces, products of eve

File-by-file Integration

Source notebook	Integrated role
Laboratorio 1	Historical log returns and GBM path simulation
Laboratorio 2	Rolling realized volatility and volatility regimes
Laboratorio 3	European option Monte Carlo vs. Black-Scholes
Laboratorio 4	Longstaff-Schwartz American put
Proyecto 4	Black-Scholes calibrated with historical volatility and compared with market option data
Untitled5	Parisian down-and-out barrier option
Untitled3	GARCH(1,1), RSI, volume, conditional volatility
Series de Tiempo	AR/ARIMA/SARIMA/Prophet and model-selection diagnostics
Calculo Estocastico	Probability sample-space exercises

Methodology

The consolidated methodology follows the modelling hierarchy used across the notebooks.

1. Historical returns and volatility calibration

The notebooks use daily adjusted or close prices and compute log returns as

$$[r_t = \ln\left(\frac{S_t}{S_{t-1}}\right).]$$

The daily sample mean (m) and standard deviation (s) are annualized as

$$[\sigma = \sqrt{252}s, \quad \mu \approx 252m + \frac{1}{2}252s^2.]$$

In these notebooks, (σ) is the annualized historical volatility. It is backward-looking and estimated from realized returns. The drift formula reconstructs an approximate price-process drift from log-return moments under a GBM convention.

2. GBM simulation

For price simulation, the notebooks use the discrete-time GBM update

$$[S_{t+\Delta t} = S_t \exp\left((\mu - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}Z_t\right), \quad Z_t \sim N(0,1).]$$

For option pricing under the risk-neutral measure (Q), the drift changes from (μ) to (r) :

$$[S_T = S_0 \exp\left((r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z\right).]$$

This is economically important: historical simulation uses estimated expected return, while pricing uses the risk-free rate because discounted payoffs are valued under (Q).

3. Rolling realized volatility

Rolling volatility is computed as the rolling standard deviation of log returns multiplied by $(\sqrt{252})$. The notebooks use 21, 63, and 126 trading-day windows to approximate one-month, one-quarter, and half-year realized volatility. Quiet and turbulent periods are defined using the 25th and 75th percentiles of the 63-day annualized volatility series.

4. Monte Carlo option pricing

For a European call or put, terminal payoffs are

$$[C_T = \max(S_T - K, 0), \quad P_T = \max(K - S_T, 0).]$$

The Monte Carlo price is the discounted sample mean:

$$[\widehat{V}_0 = e^{-rT} \frac{1}{N} \sum_{i=1}^N \text{payoff}^{(i)}.]$$

The standard error is estimated from the dispersion of discounted payoffs:

$$[SE(\widehat{V}_0) = \frac{s\{\text{discounted payoff}\}}{\sqrt{N}}.]$$

A 95% confidence interval is approximated as $(\widehat{V}_0 \pm 1.96SE)$.

5. Black-Scholes pricing and Greeks

The Black-Scholes quantities are

$$[d_1 = \frac{\ln(S_0/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}.]$$

With continuous dividend yield (q), the European call and put are

$$[C = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2),]$$

$$[P = K e^{-rT} N(-d_2) - S_0 e^{-qT} N(-d_1).]$$

The project notebook also calculates Greeks and solves implied volatility by root-finding: it finds the volatility that makes Black-Scholes equal to an observed market option price.

6. Longstaff-Schwartz American put

The American put notebook simulates risk-neutral paths and works backward through exercise dates. At each date (t), for in-the-money paths, the immediate exercise value is compared with a regression estimate of continuation value. The basis used in the source code is

$$\left[1, \frac{S_t}{K}, \left(\frac{S_t}{K} \right)^2 \right]$$

Exercise occurs when

$$[K - S_t > \widehat{C}(S_t),]$$

where $\widehat{C}(S_t)$ is the fitted continuation value. The estimated American price is the discounted mean of the resulting pathwise cashflows.

7. Parisian barrier option

The Parisian notebook prices a down-and-out option whose knock-out condition depends on consecutive time spent below a barrier (H). The clock increments when ($S_t < H$) and resets to zero otherwise. A path is knocked out when its maximum consecutive clock reaches (D). Larger (D) means the barrier is harder to trigger, so the option price should generally increase as (D) increases.

8. GARCH(1,1)

The GARCH notebook models conditional variance as

$$[\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2.]$$

(α_1) measures the short-run impact of shocks; (β_1) measures persistence of past volatility. When ($\alpha_1 + \beta_1$) is close to 1, volatility is highly persistent.

9. AR/ARIMA/SARIMA/Prophet

The ARIMA notebooks use

$$[\text{ARIMA}(p,d,q)]$$

where (p) is the autoregressive order, (d) is the number of differences, and (q) is the moving-average order. SARIMA extends this to seasonal structure:

$$[\text{SARIMA}(p,d,q)(P,D,Q)_s.]$$

AIC and BIC are used for model selection:

$$[\text{AIC} = 2k - 2\ln(L), \quad \text{BIC} = k\ln(n) - 2\ln(L).]$$

Lower values are preferred. BIC penalizes complexity more strongly than AIC when (n) is large.

Prophet is documented separately because it does not use AIC/BIC in the same way as likelihood-based ARIMA models. Its evaluation is usually based on cross-validation, error metrics, and decomposition of trend/seasonality.

10. Probability and sample spaces

The stochastic/probability exercises define finite sample spaces, Cartesian products, and events. For two dice,

[$\Omega = \{(d_1, d_2): d_1, d_2 \in \{1, \dots, 6\}\}, \quad |\Omega| = 36.]$

Bayes' theorem is included conceptually as

[$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}, \quad P(B) \neq 0.]$

Key Formulas and Financial/Statistical Interpretation

Formula	Interpretation
$(r_t = \ln(S_t/S_{t-1}))$	Continuously compounded return; addi
$(\sigma = \sqrt{252}s)$	Annualizes daily volatility under the sq
$(S_T = S_0 e^{(r - \sigma^2/2)T + \sigma \sqrt{T}Z})$	Risk-neutral terminal GBM price used
$(e^{-rT} E_Q[\text{payoff}])$	Present value of expected payoff under
(d_1, d_2)	Standardized Black-Scholes moneyness
$(AIC = 2k - 2 \ln L)$	Fit-complexity criterion; lower is better
$(BIC = k \ln n - 2 \ln L)$	Fit-complexity criterion with stronger s
$(\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2)$	GARCH conditional variance recursion
$(\alpha + \beta)$	Persistence of volatility shocks; values n
$(P(A \mid B) = P(B \mid A)P(A)/P(B))$	Updates probability of event (A) after

Documented Code Sections

Laboratorio 1: Log returns and GBM simulation

Objective. Estimate historical log-return moments for a selected ticker and simulate one year of GBM price paths.

Inputs/parameters. User-input ticker; Yahoo Finance close prices from 2021-05-01; 50 simulated paths; 252 trading days; (S_0) equal to the last observed price.

Method. The code computes log returns, annualizes drift and volatility, draws standard normal innovations, and constructs simulated paths through cumulative products of GBM gross returns.

Key formulas. $(r_t = \ln(S_t/S_{t-1}))$, $(\sigma = \sqrt{252}s)$, $(\mu = 252m + \frac{1}{2}252s^2)$, and $(S_{t+\Delta t} = S_t e^{((\mu - \sigma^2/2)\Delta t + \sigma \sqrt{\Delta t}Z_t)})$.

Code explanation. The code downloads prices, handles the case where the selected price column is a DataFrame, computes log returns, calibrates (μ) and (σ) , simulates paths vectorially, plots returns/history/simulations, and displays a summary table.

Output interpretation. The saved run used FCX and produced annualized volatility of 45.01%, a simulated one-year mean final price of \$77.85, and a one-standard-deviation terminal price interval of roughly \$48.13 to \$107.56. These are simulation summaries, not guarantees.

Assumptions/limitations. Uses historical drift for simulation under the physical measure, assumes constant drift/volatility, normal shocks, and no jumps. Results are random because no seed is set in this notebook.

Original code cell 0 from Laboratorio 1.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Setup & Data
ticker = input("Enter ticker: ").upper()
df = yf.download(ticker, start="2021-05-01")['Close']
if isinstance(df, pd.DataFrame): df = df.iloc[:, 0]

# 2. Statistics
log_returns = np.log(df / df.shift(1)).dropna()
m, s = log_returns.mean(), log_returns.std()
mu = (252 * m) + (0.5 * 252 * s**2)
sigma = np.sqrt(252) * s

# 3. Vectorized Simulation (GBM)
n_paths, n_days, dt, S0 = 50, 252, 1/252, df.iloc[-1]
Z = np.random.standard_normal((n_days, n_paths))
returns = np.exp((mu - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z)
paths = np.vstack([np.full(n_paths, S0), S0 * np.cumprod(returns, axis=0)])

# 4. Visualization
fig = plt.figure(figsize=(12, 12))

# Chart 1: Daily Log Returns
ax1 = plt.subplot(3, 1, 1)
ax1.plot(log_returns.index, log_returns, color='steelblue', lw=1, alpha=0.7, label='Log Returns')
ax1.axhline(m, color='red', ls='--', label=f'Mean: {m:.4f}')
ax1.axhline(m + s, color='orange', ls=':', label=f'+1 Std Dev: {s:.4f}')
ax1.axhline(m - s, color='orange', ls=':')
ax1.set_title(f"{ticker} Análisis de Retornos Logarítmicos", fontsize=14)
ax1.legend(loc='upper left')
ax1.grid(True, alpha=0.2)

# Chart 2: Historical + Simulation Comparison
ax2 = plt.subplot(3, 1, 2)
hist_days = np.arange(-len(df), 0)
sim_days = np.arange(0, n_days + 1)

# Plot history
ax2.plot(hist_days, df.values, color='black', lw=2, label='Historial Real')
# Plot simulation paths starting from t=0
ax2.plot(sim_days, paths, color='steelblue', alpha=0.2, lw=0.8)
ax2.plot(sim_days, np.mean(paths, axis=1), color='red', lw=2, label='Media Simulada')

ax2.axvline(0, color='grey', ls='--', alpha=0.5)
ax2.set_title(f"{ticker} Comparativa: Historial vs. Simulación", fontsize=14)

```

```

ax2.set_ylabel("Price ($)")
ax2.legend(loc='upper left')
ax2.grid(True, alpha=0.2)

# Chart 3: Detailed Paths (Zoomed)
ax3 = plt.subplot(3, 1, 3)
ax3.plot(sim_days, paths, color='steelblue', alpha=0.3, lw=1)
ax3.plot(sim_days, np.mean(paths, axis=1), color='red', lw=2.5, label='Mean Path')
ax3.set_title(f"{ticker} Detalle de Trayectorias Proyectadas (1 Año)", fontsize=14)
ax3.set_xlabel("Trading Days Forward")
ax3.set_ylabel("Price ($)")
ax3.legend()
ax3.grid(True, alpha=0.2)

plt.tight_layout()
plt.show()

# 5. Summary Table
final_prices = paths[-1, :]
yearly_mean = np.mean(final_prices)
yearly_std = np.std(final_prices)

stats_data = {
    "Metric": [
        "Current Price", "Daily Log Mean", "Daily Log Std Dev",
        "Yearly Price Mean", "Yearly Price Std Dev",
        "Yearly Expected Return ()", "Yearly Volatility ()",
        "±1 Standard Deviation",
        "±2 Standard Deviation (95%)",
        "±3 Standard Deviation (99%)"
    ],
    "Value": [
        f"${S0:.2f}", f"${m:.6f}", f"${s:.6f}",
        f"${yearly_mean:.2f}", f"${yearly_std:.2f}",
        f"${mu:.2%}", f"${sigma:.2%}",
        f"${max(0, yearly_mean - yearly_std):.2f}, ${yearly_mean + yearly_std:.2f}",
        f"${max(0, yearly_mean - 2*yearly_std):.2f}, ${yearly_mean + 2*yearly_std:.2f}",
        f"${max(0, yearly_mean - 3*yearly_std):.2f}, ${yearly_mean + 3*yearly_std:.2f}"
    ]
}

summary_df = pd.DataFrame(stats_data)
display(summary_df)

```

Laboratorio 2: Rolling realized volatility and volatility regimes

Objective. Measure how realized volatility changes through time and classify quiet versus turbulent regimes.

Inputs/parameters. User-input ticker; Yahoo Finance prices from 2021-05-01; adjusted close when available; rolling windows of 21, 63, and 126 trading days; annualization factor ($\sqrt{252}$).

Method. The code computes daily log returns, rolling standard deviations, annualizes them, then uses the 25th and 75th percentiles of 63-day rolling volatility as regime thresholds.

Key formulas. $\widehat{\sigma}_{t,w} = \sqrt{252} \operatorname{std}(r_{t-w+1}, \dots, r_t)$.

Code explanation. The code downloads prices, computes log returns, builds a rolling-volatility DataFrame, identifies quiet/turbulent periods, visualizes adjusted price/returns/volatility, and prints a summary table plus textual interpretation.

Output interpretation. The saved PLTR run reports average 63-day volatility of 63.57%, quiet threshold of 51.58%, turbulent threshold of 71.60%, and maximum 63-day volatility of 97.75% on 2025-04-25. This illustrates volatility clustering.

Assumptions/limitations. Rolling windows are descriptive and backward-looking. Quantile thresholds are sample-dependent and not structural market regimes.

Original code cell 0 from Laboratorio 2.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# --- Part 1: Stock Selection & Data Download ---
# This section handles the stock ticker selection and downloads adjusted daily prices.

ticker = input("Enter the stock ticker symbol: ").upper()

Prices = yf.download(
    ticker,
    start="2021-05-01",
    auto_adjust=False,
    progress=False
)

if "Adj Close" in Prices.columns:
    ticker_prices = Prices["Adj Close"]
else:
    ticker_prices = Prices["Close"]

if isinstance(ticker_prices, pd.DataFrame):
    ticker_prices = ticker_prices.iloc[:, 0]

ticker_prices = ticker_prices.dropna()
```



```

# --- Part 2: Daily Log Returns ---
# This section calculates daily logarithmic returns from adjusted prices.

log_returns = np.log(ticker_prices / ticker_prices.shift(1)).dropna()

# --- Part 3: Rolling Volatility Windows ---
# This section calculates rolling realized volatility using 21, 63, and 126 day windows.
# Standard annualization for daily returns: multiply by sqrt(252).

annualization_factor = np.sqrt(252)

windows = [21, 63, 126]

rolling_volatility = pd.DataFrame(index=log_returns.index)

for window in windows:
    rolling_volatility[f"{window}-Day Volatility"] = (
        log_returns.rolling(window=window).std() * annualization_factor
    )

# --- Part 4: Quiet and Turbulent Periods ---
# This section classifies quiet and turbulent periods using volatility quantiles.

main_window = 63
main_vol = rolling_volatility[f"{main_window}-Day Volatility"].dropna()

quiet_threshold = main_vol.quantile(0.25)
turbulent_threshold = main_vol.quantile(0.75)

quiet_periods = main_vol[main_vol <= quiet_threshold]
turbulent_periods = main_vol[main_vol >= turbulent_threshold]

avg_quiet_vol = quiet_periods.mean()
avg_turbulent_vol = turbulent_periods.mean()

max_vol = main_vol.max()
max_vol_date = main_vol.idxmax()

# --- Part 5: Visualization ---
# This section plots adjusted prices, log returns, and annualized rolling volatility.

fig = plt.figure(figsize=(12, 12))

# Adjusted prices

```

```

ax1 = plt.subplot(3, 1, 1)
ax1.plot(ticker_prices.index, ticker_prices, color="black", lw=1.5, label="Adjusted Price")
ax1.set_title(f"{ticker} Adjusted Daily Prices", fontsize=14)
ax1.set_ylabel("Price")
ax1.legend(loc="upper left")
ax1.grid(True, alpha=0.2)

# Daily log returns
ax2 = plt.subplot(3, 1, 2)
ax2.plot(log_returns.index, log_returns, color="steelblue", lw=1, alpha=0.7, label="Daily Log R")
ax2.axhline(log_returns.mean(), color="red", ls="--", label="Mean")
ax2.set_title(f"{ticker} Daily Log Returns", fontsize=14)
ax2.set_ylabel("Log Return")
ax2.legend(loc="upper left")
ax2.grid(True, alpha=0.2)

# Rolling volatility
ax3 = plt.subplot(3, 1, 3)

for column in rolling_volatility.columns:
    ax3.plot(
        rolling_volatility.index,
        rolling_volatility[column],
        lw=1.4,
        label=column
    )

ax3.axhline(
    quiet_threshold,
    color="green",
    ls="--",
    label=f"Quiet Threshold, 63-Day: {quiet_threshold:.2%}"
)

ax3.axhline(
    turbulent_threshold,
    color="purple",
    ls="--",
    label=f"Turbulent Threshold, 63-Day: {turbulent_threshold:.2%}"
)

ax3.scatter(
    max_vol_date,
    max_vol,
    color="black",
    zorder=5,
    label=f"Max 63-Day Vol: {max_vol:.2%}"
)

```

```

ax3.set_title(f"{ticker} Annualized Rolling Realized Volatility", fontsize=14)
ax3.set_xlabel("Date")
ax3.set_ylabel("Annualized Volatility")
ax3.legend(loc="upper left")
ax3.grid(True, alpha=0.2)

```

```

plt.tight_layout()
plt.show()

```

--- Part 6: Summary Table ---

This section reports descriptive statistics for the realized volatility series.

```

summary_data = {
    "Metric": [
        "Ticker",
        "Start Date",
        "End Date",
        "Annualization Factor",
        "Average 21-Day Volatility",
        "Average 63-Day Volatility",
        "Average 126-Day Volatility",
        "Quiet Threshold, 63-Day",
        "Turbulent Threshold, 63-Day",
        "Average Quiet Volatility",
        "Average Turbulent Volatility",
        "Maximum 63-Day Volatility",
        "Date of Maximum 63-Day Volatility"
    ],
    "Value": [
        ticker,
        ticker_prices.index.min().strftime("%Y-%m-%d"),
        ticker_prices.index.max().strftime("%Y-%m-%d"),
        "sqrt(252)",
        f"{rolling_volatility['21-Day Volatility'].mean():.2%}",
        f"{rolling_volatility['63-Day Volatility'].mean():.2%}",
        f"{rolling_volatility['126-Day Volatility'].mean():.2%}",
        f"{quiet_threshold:.2%}",
        f"{turbulent_threshold:.2%}",
        f"{avg_quiet_vol:.2%}",
        f"{avg_turbulent_vol:.2%}",
        f"{max_vol:.2%}",
        max_vol_date.strftime("%Y-%m-%d")
    ]
}

```

```

summary_df = pd.DataFrame(summary_data)

```

```
display(summary_df)
```

```
# --- Part 7: Interpretation ---
```

```
# This section gives a short financial interpretation of volatility clustering.
```

```
print("\nInterpretation:")
```

```
print(
```

```
    f"For {ticker}, realized volatility changes significantly through time. "
    f"Using the 63-day rolling window, quiet periods are those below approximately "
    f"{quiet_threshold:.2%}, while turbulent periods are those above approximately "
    f"{turbulent_threshold:.2%}. The maximum 63-day annualized volatility was "
    f"{max_vol:.2%} on {max_vol_date.strftime('%Y-%m-%d')}."
)
```

```
print(
```

```
    "\nThe graph shows volatility clustering: periods of high volatility tend to be followed "
    "by more high-volatility periods, while calm periods tend to persist as well. "
    "This means that market risk is not evenly distributed through time. Instead, risk appears "
    "in clusters, especially around turbulent market episodes."
)
```

Laboratorio 3: European option Monte Carlo and Black-Scholes comparison

Objective. Price a European call or put by Monte Carlo under risk-neutral GBM and compare it with the Black-Scholes closed-form price.

Inputs/parameters. User-input ticker, option type, strike (K), maturity (T), risk-free rate (r), and number of paths. Volatility is estimated from historical log returns.

Method. The code simulates terminal prices (S_T) directly under the risk-neutral measure, computes discounted payoffs, estimates a Monte Carlo standard error and confidence interval, and calculates Black-Scholes (d_1, d_2) and price.

Key formulas. ($S_T = S_0 e^{\{(r - \sigma^2/2)T + \sigma \sqrt{T}Z\}}$), ($\widehat{V}_0 = e^{-rT} \bar{X}$), and the standard Black-Scholes call/put formulas.

Code explanation. The code downloads prices, estimates (σ), asks for option parameters, simulates terminal prices with `np.random.seed(42)`, prices the option, plots returns/terminal distribution/convergence, and displays a comparison table.

Output interpretation. The saved PLTR call run used ($K=350$), ($T=3$), ($r=0.60\%$), and 1,000,000 paths. Monte Carlo price was \$25.1699 versus Black-Scholes price \$25.3885, a -0.86% difference, consistent with sampling error under the same model.

Assumptions/limitations. The market price is not used. The volatility is historical, constant, and backward-looking. The option is European only; early exercise and smile/skew are excluded.

Original code cell 0 from Laboratorio 3.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# 1. Setup & Data
ticker = input("Enter ticker: ").upper()
option_type = input("Call or Put? ").lower()

Prices = yf.download(ticker, start="2021-05-01", auto_adjust=True)
df = Prices["Close"]

if isinstance(df, pd.DataFrame):
    df = df.iloc[:, 0]

S0 = df.iloc[-1]

# Inputs de la opción
K_input = input(f"Enter strike K [default = current price {S0:.2f}]: ")
K = float(K_input) if K_input.strip() != "" else S0

T = float(input("Enter time to maturity T in years, e.g. 1: "))
r = float(input("Enter annual risk-free rate r, e.g. 0.05: "))
n_paths = int(input("Enter number of Monte Carlo paths, e.g. 10000: "))

np.random.seed(42)

# 2. Historical Volatility Estimation
log_returns = np.log(df / df.shift(1)).dropna()

m = log_returns.mean()
s = log_returns.std()

mu = (252 * m) + (0.5 * 252 * s**2)
sigma = np.sqrt(252) * s

# 3. Monte Carlo Simulation Under Risk-Neutral Measure
Z = np.random.standard_normal(n_paths)

ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)

if option_type == "call":
    payoffs = np.maximum(ST - K, 0)
elif option_type == "put":
    payoffs = np.maximum(K - ST, 0)
else:
    raise ValueError("option_type must be 'call' or 'put'.")

```

```

discounted_payoffs = np.exp(-r * T) * payoffs

mc_price = np.mean(discounted_payoffs)
mc_std = np.std(discounted_payoffs, ddof=1)
mc_se = mc_std / np.sqrt(n_paths)

ci_low = mc_price - 1.96 * mc_se
ci_high = mc_price + 1.96 * mc_se

# 4. Black-Scholes Closed Form
d1 = (np.log(S0 / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
d2 = d1 - sigma * np.sqrt(T)

if option_type == "call":
    bs_price = S0 * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
else:
    bs_price = K * np.exp(-r * T) * norm.cdf(-d2) - S0 * norm.cdf(-d1)

pricing_error = mc_price - bs_price
pricing_error_pct = pricing_error / bs_price

# 5. Visualization
fig = plt.figure(figsize=(12, 12))

# Chart 1: Daily Log Returns
ax1 = plt.subplot(3, 1, 1)
ax1.plot(log_returns.index, log_returns, color="steelblue", lw=1, alpha=0.7, label="Log Returns")
ax1.axhline(m, color="red", ls="--", label=f"Mean: {m:.6f}")
ax1.axhline(m + s, color="orange", ls=":", label=f"+1 Std Dev: {s:.6f}")
ax1.axhline(m - s, color="orange", ls=":")
ax1.set_title(f"{ticker} Análisis de Retornos Logarítmicos", fontsize=14)
ax1.legend(loc="upper left")
ax1.grid(True, alpha=0.2)

# Chart 2: Simulated Terminal Prices
ax2 = plt.subplot(3, 1, 2)
ax2.hist(ST, bins=60, alpha=0.75, edgecolor="black")
ax2.axvline(S0, color="black", lw=2, label=f"S0: {S0:.2f}")
ax2.axvline(K, color="red", ls="--", lw=2, label=f"K: {K:.2f}")
ax2.axvline(np.mean(ST), color="steelblue", ls=":", lw=2, label=f"Mean ST: {np.mean(ST):.2f}")
ax2.set_title(f"{ticker} Distribución Simulada de $$T$ Bajo Medida Neutral al Riesgo", fontsize=14)
ax2.set_xlabel("Terminal Price")
ax2.set_ylabel("Frequency")
ax2.legend(loc="upper right")
ax2.grid(True, alpha=0.2)

# Chart 3: Monte Carlo Price Convergence

```

```

running_price = np.exp(-r * T) * np.cumsum(payoffs) / np.arange(1, n_paths + 1)

ax3 = plt.subplot(3, 1, 3)
ax3.plot(running_price, color="steelblue", lw=1, alpha=0.8, label="Monte Carlo Running Price")
ax3.axhline(mc_price, color="red", ls="--", lw=2, label=f"MC Price: {mc_price:.4f}")
ax3.axhline(bs_price, color="black", ls=":", lw=2, label=f"Black-Scholes: {bs_price:.4f}")
ax3.set_title(f"{ticker} Convergencia del Precio Monte Carlo", fontsize=14)
ax3.set_xlabel("Number of Simulations")
ax3.set_ylabel("Option Price")
ax3.legend(loc="upper right")
ax3.grid(True, alpha=0.2)

plt.tight_layout()
plt.show()

```

6. Summary Table

```

stats_data = {
    "Metric": [
        "Ticker",
        "Option Type",
        "Current Price S0",
        "Strike K",
        "Time to Maturity T",
        "Risk-Free Rate r",
        "Number of Paths",
        "Daily Log Mean",
        "Daily Log Std Dev",
        "Historical Expected Return ",
        "Historical Volatility ",
        "Monte Carlo Price",
        "Monte Carlo Standard Error",
        "95% Confidence Interval",
        "Black-Scholes Price",
        "MC - BS Difference",
        "MC - BS Difference (%)"
    ],
    "Value": [
        ticker,
        option_type.capitalize(),
        f"${S0:.2f}",
        f"${K:.2f}",
        f"{T:.2f} years",
        f"{r:.2%}",
        f"{n_paths:,}",
        f"{m:.6f}",
        f"{s:.6f}",
        f"{mu:.2%}",
        f"{sigma:.2%}",
    ]
}

```

```

        f"${mc_price:.4f}",
        f"${mc_se:.4f}",
        f"[${ci_low:.4f}, ${ci_high:.4f}]",
        f"${bs_price:.4f}",
        f"${pricing_error:.4f}",
        f"${pricing_error_pct:.2%}"
    ]
}

summary_df = pd.DataFrame(stats_data)
display(summary_df)

# 7. Interpretation
print("\nInterpretation:")
print(f"The estimated annualized historical volatility is {sigma:.2%}.")
print(f"The Monte Carlo estimated {option_type} price is ${mc_price:.4f}.")
print(f"The 95% confidence interval is [{ci_low:.4f}, {ci_high:.4f}].")
print(f"The Black-Scholes closed-form price is ${bs_price:.4f}.")
print(f"The Monte Carlo estimate differs from Black-Scholes by ${pricing_error:.4f}, or {pricing_error_pct:.2%}.")

if abs(pricing_error_pct) < 0.02:
    print("The Monte Carlo estimate is very close to the Black-Scholes price, which is expected.")
else:
    print("The Monte Carlo estimate differs more visibly from Black-Scholes. This can happen with low volatility or short maturities.")

```

Laboratorio 4: American put by Longstaff-Schwartz

Objective. Estimate the price of an American put and quantify early-exercise behavior.

Inputs/parameters. User-input ticker, strike (K), maturity (T), risk-free rate (r), number of paths, and number of time steps. Volatility is estimated from daily log returns.

Method. The code simulates full risk-neutral paths and performs backward induction. On in-the-money paths, discounted future cashflows are regressed on polynomial basis functions of (S_t/K) . Immediate exercise is chosen when intrinsic value exceeds estimated continuation value.

Key formulas. Payoff $(=\max(K-S_t, 0))$. Basis $(1, S_t/K, (S_t/K)^2)$. Exercise rule $(K-S_t > \widehat{C}(S_t))$.

Code explanation. The code builds the simulated path matrix, computes the payoff matrix, loops backward through time, discounts cashflows, fits least squares continuation values, records exercise decisions and an approximate boundary, then compares against European Black-Scholes and European Monte Carlo.

Output interpretation. The saved PLTR run reports American put LSM price \$58.8647, European Black-Scholes put \$51.7611, European MC put \$52.0694, early-exercise premium \$7.1036, and early exercise on 67.59% of simulated paths. This is economically consistent with a put where early exercise may be valuable.

Assumptions/limitations. Regression basis is simple; LSM has simulation error and regression approximation error. Exercise boundary is approximate. The code enforces American value not

below immediate exercise value.

Original code cell 0 from Laboratorio 4.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# 1. Setup & Data
ticker = input("Enter ticker: ").upper()

Prices = yf.download(ticker, start="2021-05-01", auto_adjust=True)
df = Prices["Close"]

if isinstance(df, pd.DataFrame):
    df = df.iloc[:, 0]

df = df.dropna()

if len(df) < 252:
    raise ValueError("Not enough data. Use at least one year of daily prices.")

S0 = float(df.iloc[-1])

# Option parameters
K_input = input(f"Enter strike K [default: {S0:.2f}]: ")
T_input = input("Enter maturity T in years [default: 1]: ")
r_input = input("Enter risk-free rate r [default: 0.04]: ")
paths_input = input("Enter number of simulation paths [default: 20000]: ")
steps_input = input("Enter number of time steps [default: 252]: ")

K = float(K_input) if K_input.strip() else S0
T = float(T_input) if T_input.strip() else 1.0
r = float(r_input) if r_input.strip() else 0.04
n_paths = int(paths_input) if paths_input.strip() else 20000
n_steps = int(steps_input) if steps_input.strip() else 252

np.random.seed(42)

# 2. Statistics
log_returns = np.log(df / df.shift(1)).dropna()

m = log_returns.mean()
s = log_returns.std()
```

```

mu = (252 * m) + (0.5 * 252 * s**2)
sigma = np.sqrt(252) * s

if sigma <= 0:
    raise ValueError("Estimated volatility is not positive. Check the data.")

# 3. Vectorized Simulation Under Q
dt = T / n_steps
discount = np.exp(-r * dt)

Z = np.random.standard_normal((n_steps, n_paths))

returns_Q = np.exp(
    (r - 0.5 * sigma**2) * dt
    + sigma * np.sqrt(dt) * Z
)

paths = np.vstack([
    np.full(n_paths, S0),
    S0 * np.cumprod(returns_Q, axis=0)
])

time_grid = np.linspace(0, T, n_steps + 1)

# Payoff matrix for an American put
payoff = np.maximum(K - paths, 0)

# 4. Longstaff-Schwartz Backward Regression
cashflows = payoff[-1].copy()
exercise_time = np.full(n_paths, n_steps)

exercise_boundary = np.full(n_steps + 1, np.nan)
snapshot = None
snapshot_step = n_steps // 2

for t in range(n_steps - 1, 0, -1):

    # Discount future cashflows one step back
    cashflows *= discount

    # Only in-the-money paths are relevant for early exercise decision
    in_the_money = payoff[t] > 0

    if np.sum(in_the_money) >= 5:

        S_t = paths[t, in_the_money]
        Y = cashflows[in_the_money]

```

```

# Basis functions: 1, S_t, S_t^2
# Scaling by K improves numerical stability but preserves the polynomial basis.
X = S_t / K
A = np.column_stack([
    np.ones_like(X),
    X,
    X**2
])

beta = np.linalg.lstsq(A, Y, rcond=None)[0]
continuation_value = A @ beta

exercise_value = payoff[t, in_the_money]
exercise_now = exercise_value > continuation_value

exercise_indices = np.where(in_the_money)[0][exercise_now]

if len(exercise_indices) > 0:
    cashflows[exercise_indices] = payoff[t, exercise_indices]
    exercise_time[exercise_indices] = t
    exercise_boundary[t] = np.max(paths[t, exercise_indices])

# Save one regression snapshot for visualization
if t == snapshot_step:
    snapshot = {
        "S_t": S_t.copy(),
        "Y": Y.copy(),
        "continuation": continuation_value.copy(),
        "exercise": exercise_value.copy(),
        "beta": beta.copy(),
        "t": t
    }

# Discount from first step to time zero
american_samples = cashflows * discount
american_price = np.mean(american_samples)

# American value cannot be below immediate exercise value
immediate_value_0 = max(K - S0, 0)
american_price = max(american_price, immediate_value_0)

american_std_error = np.std(american_samples, ddof=1) / np.sqrt(n_paths)

# 5. European Put Benchmark
d1 = (np.log(S0 / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
d2 = d1 - sigma * np.sqrt(T)

```

```

european_put_bs = K * np.exp(-r * T) * norm.cdf(-d2) - S0 * norm.cdf(-d1)

european_payoff_mc = np.maximum(K - paths[-1], 0)
european_put_mc = np.exp(-r * T) * np.mean(european_payoff_mc)
european_std_error = (
    np.exp(-r * T)
    * np.std(european_payoff_mc, ddof=1)
    / np.sqrt(n_paths)
)

early_exercise_mask = exercise_time < n_steps
early_exercise_fraction = np.mean(early_exercise_mask)
early_exercise_premium = american_price - european_put_bs

# 6. Visualization
fig = plt.figure(figsize=(12, 16))

# Chart 1: Daily Log Returns
ax1 = plt.subplot(4, 1, 1)

ax1.plot(
    log_returns.index,
    log_returns,
    color="steelblue",
    lw=1,
    alpha=0.7,
    label="Log Returns"
)

ax1.axhline(m, color="red", ls="--", label=f"Mean: {m:.4f}")
ax1.axhline(m + s, color="orange", ls=":", label=f"+1 Std Dev: {s:.4f}")
ax1.axhline(m - s, color="orange", ls=":")

ax1.set_title(f"{ticker} Análisis de Retornos Logarítmicos", fontsize=14)
ax1.legend(loc="upper left")
ax1.grid(True, alpha=0.2)

# Chart 2: Simulated Risk-Neutral Paths
ax2 = plt.subplot(4, 1, 2)

plot_paths = min(80, n_paths)

ax2.plot(
    time_grid,
    paths[:, :plot_paths],
    color="steelblue",
    alpha=0.18,
    lw=0.8

```

```

)

ax2.plot(
    time_grid,
    np.mean(paths, axis=1),
    color="red",
    lw=2,
    label="Media Simulada bajo Q"
)

ax2.axhline(K, color="black", ls="--", lw=1.5, label=f"Strike K = {K:.2f}")

ax2.set_title(f"{ticker} Trayectorias Simuladas bajo Medida Neutral al Riesgo Q", fontsize=14)
ax2.set_ylabel("Price ($)")
ax2.legend(loc="upper left")
ax2.grid(True, alpha=0.2)

# Chart 3: Approximate Early Exercise Boundary
ax3 = plt.subplot(4, 1, 3)

ax3.plot(
    time_grid,
    exercise_boundary,
    color="purple",
    lw=2,
    label="Frontera aproximada de ejercicio"
)

ax3.axhline(K, color="black", ls="--", lw=1.5, label="Strike K")

ax3.set_title("Put Americana: Frontera Aproximada de Ejercicio Temprano", fontsize=14)
ax3.set_xlabel("Time to Maturity")
ax3.set_ylabel("Underlying Price")
ax3.legend(loc="upper left")
ax3.grid(True, alpha=0.2)

# Chart 4: Regression Snapshot or Exercise Histogram
ax4 = plt.subplot(4, 1, 4)

if snapshot is not None:
    S_snap = snapshot["S_t"]
    Y_snap = snapshot["Y"]
    exercise_snap = snapshot["exercise"]
    beta_snap = snapshot["beta"]

    sample_size = min(1500, len(S_snap))
    sample_idx = np.random.choice(len(S_snap), size=sample_size, replace=False)

```

```

S_plot = S_snap[sample_idx]
Y_plot = Y_snap[sample_idx]

S_line = np.linspace(S_snap.min(), S_snap.max(), 300)
X_line = S_line / K

A_line = np.column_stack([
    np.ones_like(X_line),
    X_line,
    X_line**2
])

continuation_line = A_line @ beta_snap
exercise_line = np.maximum(K - S_line, 0)

ax4.scatter(
    S_plot,
    Y_plot,
    alpha=0.25,
    s=10,
    color="steelblue",
    label="Cashflows descontados"
)

ax4.plot(
    S_line,
    continuation_line,
    color="red",
    lw=2.5,
    label="Valor de continuación estimado"
)

ax4.plot(
    S_line,
    exercise_line,
    color="black",
    lw=2,
    ls="--",
    label="Valor de ejercicio inmediato"
)

ax4.set_title(
    f"Regresión Longstaff-Schwartz en  $t = \{\text{snapshot['t']}\}$  pasos",
    fontsize=14
)

ax4.set_xlabel("Underlying Price")
ax4.set_ylabel("Value")

```

```

ax4.legend(loc="upper right")
ax4.grid(True, alpha=0.2)

else:
    early_steps = exercise_time[early_exercise_mask]

    if len(early_steps) > 0:
        ax4.hist(
            early_steps,
            bins=30,
            color="steelblue",
            alpha=0.75,
            edgecolor="black"
        )
        ax4.set_title("Distribución de Fechas de Ejercicio Temprano", fontsize=14)
        ax4.set_xlabel("Time Step")
        ax4.set_ylabel("Number of Paths")
        ax4.grid(True, alpha=0.2)
    else:
        ax4.text(
            0.5,
            0.5,
            "No early exercise detected in this simulation.",
            ha="center",
            va="center",
            fontsize=13
        )
        ax4.set_axis_off()

plt.tight_layout()
plt.show()

# 7. Summary Table
if early_exercise_premium > 2 * american_std_error:
    exercise_comment = (
        "La put americana muestra una prima positiva frente a la europea. "
        "Esto es consistente con el valor económico del ejercicio temprano."
    )
elif early_exercise_fraction > 0:
    exercise_comment = (
        "El algoritmo detecta ejercicio temprano en algunas trayectorias, "
        "pero la prima frente a la europea es pequeña con estos parámetros."
    )
else:
    exercise_comment = (
        "No se observa ejercicio temprano relevante. Bajo estos parámetros, "
        "la put americana se comporta de forma cercana a la europea."
    )

```

```

stats_data = {
    "Metric": [
        "Current Price S0",
        "Strike K",
        "Maturity T",
        "Risk-Free Rate r",
        "Daily Log Mean",
        "Daily Log Std Dev",
        "Yearly Expected Return ()",
        "Yearly Volatility ()",
        "Simulation Paths",
        "Time Steps",
        "American Put Price - LSM",
        "American Std Error",
        "European Put Price - Black-Scholes",
        "European Put Price - Monte Carlo",
        "European MC Std Error",
        "Early Exercise Premium",
        "Early Exercise Paths",
        "Early Exercise %"
    ],
    "Value": [
        f"${S0:.2f}",
        f"${K:.2f}",
        f"${T:.2f} years",
        f"${r:.2%}",
        f"${m:.6f}",
        f"${s:.6f}",
        f"${mu:.2%}",
        f"${sigma:.2%}",
        f"${n_paths:,:}",
        f"${n_steps:,:}",
        f"${american_price:.4f}",
        f"${american_std_error:.4f}",
        f"${european_put_bs:.4f}",
        f"${european_put_mc:.4f}",
        f"${european_std_error:.4f}",
        f"${early_exercise_premium:.4f}",
        f"${early_exercise_mask.sum():,:}",
        f"${early_exercise_fraction:.2%}"
    ]
}

summary_df = pd.DataFrame(stats_data)
display(summary_df)

print("\nComentario sobre ejercicio temprano:")

```



```
print(exercise_comment)
```

Proyecto 4: Black-Scholes with historical volatility and market option data

Objective. Evaluate how reasonable a Black-Scholes price is when volatility is calibrated from historical returns and compared with listed option-market data.

Inputs/parameters. Stock/ETF ticker, option type, risk-free rate (r), continuous dividend yield (q), Yahoo Finance option expiration, and strike (K). If option-chain data are unavailable, maturity (T) is entered manually.

Method. The code downloads historical prices and option chains, selects a listed strike, estimates historical volatility, prices the option with Black-Scholes, computes Greeks, extracts bid/ask/last/Yahoo implied volatility, and solves implied volatility from the chosen market price.

Key formulas. Dividend-adjusted Black-Scholes call/put, Greeks, and implied volatility root ($BS(\sigma_{\text{imp}}) - P_{\text{market}} = 0$).

Code explanation. The code defines reusable functions for price, Greeks, and implied volatility, then visualizes price history, log returns, historical vs implied volatility, and theoretical vs market price.

Output interpretation. The saved SMCi call run used ($S_0=35.58$, $K=41.00$), ($T=0.0082$), ($r=5\%$), historical ($\sigma=84.05\%$). Black-Scholes price was \$0.0361, market last price \$0.2200, and implied volatility from market price 125.03%. The model price was 83.60% below the market price.

Assumptions/limitations. Market option data can be stale, illiquid, or affected by zero bid/ask quotes. A Black-Scholes difference is not automatically arbitrage. Historical volatility is backward-looking; implied volatility is forward-looking.

Original code cell 0 from Proyecto 4.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from scipy.stats import norm
from scipy.optimize import brentq
from IPython.display import display

# =====
# PROYECTO 4: BLACK-SCHOLES CON DATOS REALES
# Pregunta central:
# ¿Qué tan razonable es un precio Black-Scholes cuando se calibra
# la volatilidad con datos históricos?
# =====
```

```

# -----
# 1. Selección del activo y descarga de precios
# -----

stock_ticker = input("Enter stock or ETF ticker [default: SPY]: ").upper() or "SPY"
option_type = input("Call or Put? [default: call]: ").lower() or "call"

if option_type not in ["call", "put"]:
    raise ValueError("option_type must be either 'call' or 'put'.")

Prices = yf.download(
    stock_ticker,
    start="2021-05-01",
    auto_adjust=True,
    progress=False
)

if Prices.empty:
    raise ValueError("No price data was downloaded. Check the ticker symbol.")

df = Prices["Close"]

if isinstance(df, pd.DataFrame):
    df = df.iloc[:, 0]

df = df.dropna()

if len(df) < 252:
    raise ValueError("Not enough data. Use at least one year of daily prices.")

S0 = float(df.iloc[-1])

r_input = input("Enter annual risk-free rate r [default: 0.045]: ")
q_input = input("Enter continuous dividend yield q [default: 0.000]: ")

r = float(r_input) if r_input.strip() else 0.045
q = float(q_input) if q_input.strip() else 0.000

# -----
# 2. Búsqueda de cadena de opciones en Yahoo Finance
# -----

yticker = yf.Ticker(stock_ticker)

try:
    available_expirations = list(yticker.options)
except Exception:

```

```

    available_expirations = []

selected_expiration = None
option_chain = None
chain_df = pd.DataFrame()
market_row = None
market_price_source = "Not available"

if len(available_expirations) > 0:
    print("\nAvailable expirations from Yahoo Finance:")
    print(available_expirations[:12])

    expiration_input = input(
        "Enter expiration YYYY-MM-DD [blank = nearest available expiration]: "
    )

    if expiration_input.strip():
        if expiration_input.strip() in available_expirations:
            selected_expiration = expiration_input.strip()
        else:
            selected_expiration = available_expirations[0]
            print(
                f"Expiration not found. Using nearest available expiration: {selected_expiration}"
            )
    else:
        selected_expiration = available_expirations[0]

    option_chain = yticker.option_chain(selected_expiration)
    chain_df = option_chain.calls if option_type == "call" else option_chain.puts

    expiration_date = pd.Timestamp(selected_expiration)
    today = pd.Timestamp.today().normalize()
    T = max((expiration_date - today).days / 365, 1 / 365)

else:
    print("\nNo option chain was available from Yahoo Finance for this ticker.")
    T_input = input("Enter maturity T in years [default: 1.0]: ")
    T = float(T_input) if T_input.strip() else 1.0

# -----
# 3. Selección del strike K
# -----

K_input = input(f"Enter strike K [blank = closest listed strike to S0 = {S0:.2f}]: ")

if len(chain_df) > 0 and "strike" in chain_df.columns:
    available_strikes = chain_df["strike"].astype(float).values

```

```

    if K_input.strip():
        requested_K = float(K_input)
        nearest_idx = np.abs(available_strikes - requested_K).argmin()
        K = float(available_strikes[nearest_idx])

        if abs(K - requested_K) > 1e-8:
            print(
                f"Requested strike {requested_K:.2f} was not listed. "
                f"Using nearest listed strike: {K:.2f}"
            )
        else:
            nearest_idx = np.abs(available_strikes - S0).argmin()
            K = float(available_strikes[nearest_idx])

    market_row = chain_df.iloc[np.abs(chain_df["strike"].astype(float) - K).argmin()]

else:
    K = float(K_input) if K_input.strip() else S0

# -----
# 4. Estimación de volatilidad histórica
# -----

log_returns = np.log(df / df.shift(1)).dropna()

m = log_returns.mean()
s = log_returns.std()

# Retorno esperado anualizado bajo GBM:
# E[log-return diario] se transforma a drift aproximado del precio.
mu = (252 * m) + (0.5 * 252 * s**2)

# Volatilidad histórica anualizada estándar:
sigma = np.sqrt(252) * s

if sigma <= 0:
    raise ValueError("Estimated historical volatility is not positive. Check the data.")

# -----
# 5. Funciones Black-Scholes
# -----

def black_scholes_price(S, K, T, r, sigma, option_type="call", q=0.0):
    """
    European Black-Scholes price with optional continuous dividend yield q.

```

```

S: current underlying price
K: strike
T: maturity in years
r: continuously compounded risk-free rate
sigma: annualized volatility
option_type: 'call' or 'put'
q: continuous dividend yield
"""

if T <= 0:
    return max(S - K, 0) if option_type == "call" else max(K - S, 0)

if sigma <= 0:
    forward = S * np.exp((r - q) * T)
    payoff = max(forward - K, 0) if option_type == "call" else max(K - forward, 0)
    return np.exp(-r * T) * payoff

d1 = (np.log(S / K) + (r - q + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
d2 = d1 - sigma * np.sqrt(T)

if option_type == "call":
    price = S * np.exp(-q * T) * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
else:
    price = K * np.exp(-r * T) * norm.cdf(-d2) - S * np.exp(-q * T) * norm.cdf(-d1)

return price

def black_scholes_greeks(S, K, T, r, sigma, option_type="call", q=0.0):
    """
    European option Greeks.
    Vega and Rho are reported per 1.00 change, not per 1%.
    """

    d1 = (np.log(S / K) + (r - q + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)

    pdf_d1 = norm.pdf(d1)

    if option_type == "call":
        delta = np.exp(-q * T) * norm.cdf(d1)
        theta = (
            -S * np.exp(-q * T) * pdf_d1 * sigma / (2 * np.sqrt(T))
            - r * K * np.exp(-r * T) * norm.cdf(d2)
            + q * S * np.exp(-q * T) * norm.cdf(d1)
        )
        rho = K * T * np.exp(-r * T) * norm.cdf(d2)

```

```

else:
    delta = -np.exp(-q * T) * norm.cdf(-d1)
    theta = (
        -S * np.exp(-q * T) * pdf_d1 * sigma / (2 * np.sqrt(T))
        + r * K * np.exp(-r * T) * norm.cdf(-d2)
        - q * S * np.exp(-q * T) * norm.cdf(-d1)
    )
    rho = -K * T * np.exp(-r * T) * norm.cdf(-d2)

gamma = np.exp(-q * T) * pdf_d1 / (S * sigma * np.sqrt(T))
vega = S * np.exp(-q * T) * pdf_d1 * np.sqrt(T)

return {
    "Delta": delta,
    "Gamma": gamma,
    "Vega": vega,
    "Theta / Year": theta,
    "Theta / Day": theta / 252,
    "Rho": rho
}

def implied_volatility_from_price(price, S, K, T, r, option_type="call", q=0.0):
    """
    Solves for implied volatility using the observed option price.
    """

    if price is None or not np.isfinite(price) or price <= 0 or T <= 0:
        return np.nan

    def objective(vol):
        return black_scholes_price(S, K, T, r, vol, option_type, q) - price

    try:
        return brentq(objective, 1e-6, 5.0, maxiter=500)
    except ValueError:
        return np.nan

# -----
# 6. Precio teórico Black-Scholes con volatilidad histórica
# -----

bs_price = black_scholes_price(S0, K, T, r, sigma, option_type, q)

d1 = (np.log(S0 / K) + (r - q + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
d2 = d1 - sigma * np.sqrt(T)

```

```

greeks = black_scholes_greeks(S0, K, T, r, sigma, option_type, q)

# -----
# 7. Precio de mercado e implied volatility
# -----

market_bid = np.nan
market_ask = np.nan
market_last = np.nan
market_mid = np.nan
yahoo_iv = np.nan
market_price = np.nan
implied_vol_market = np.nan
pricing_difference = np.nan
pricing_difference_pct = np.nan

if market_row is not None:
    market_bid = float(market_row.get("bid", np.nan))
    market_ask = float(market_row.get("ask", np.nan))
    market_last = float(market_row.get("lastPrice", np.nan))
    yahoo_iv = float(market_row.get("impliedVolatility", np.nan))

    if (
        np.isfinite(market_bid)
        and np.isfinite(market_ask)
        and market_bid > 0
        and market_ask > 0
    ):
        market_mid = (market_bid + market_ask) / 2
        market_price = market_mid
        market_price_source = "Bid-Ask Mid"

    elif np.isfinite(market_last) and market_last > 0:
        market_price = market_last
        market_price_source = "Last Price"

    implied_vol_market = implied_volatility_from_price(
        market_price, S0, K, T, r, option_type, q
    )

    if np.isfinite(market_price):
        pricing_difference = bs_price - market_price
        pricing_difference_pct = pricing_difference / market_price

# -----
# 8. Gráficas

```

```

# -----

fig = plt.figure(figsize=(12, 16))

# Gráfica 1: precios ajustados
ax1 = plt.subplot(4, 1, 1)
ax1.plot(df.index, df, color="black", lw=1.5, label="Adjusted Close")
ax1.axhline(K, color="red", ls="--", lw=1.5, label=f"Strike K = {K:.2f}")
ax1.scatter(df.index[-1], S0, color="steelblue", zorder=5, label=f"S0 = {S0:.2f}")
ax1.set_title(f"{stock_ticker} Adjusted Prices and Selected Strike", fontsize=14)
ax1.set_ylabel("Price")
ax1.legend(loc="upper left")
ax1.grid(True, alpha=0.2)

# Gráfica 2: log-retornos diarios
ax2 = plt.subplot(4, 1, 2)
ax2.plot(
    log_returns.index,
    log_returns,
    color="steelblue",
    lw=1,
    alpha=0.7,
    label="Daily Log Returns"
)
ax2.axhline(m, color="red", ls="--", label=f"Mean: {m:.6f}")
ax2.axhline(m + s, color="orange", ls=":", label=f"+1 Std Dev: {s:.6f}")
ax2.axhline(m - s, color="orange", ls=":")
ax2.set_title(f"{stock_ticker} Daily Log Returns", fontsize=14)
ax2.set_ylabel("Log Return")
ax2.legend(loc="upper left")
ax2.grid(True, alpha=0.2)

# Gráfica 3: volatilidad histórica vs implícita
ax3 = plt.subplot(4, 1, 3)
vol_labels = ["Historical "]
vol_values = [sigma]

if np.isfinite(implied_vol_market):
    vol_labels.append("Implied from Market Price")
    vol_values.append(implied_vol_market)
elif np.isfinite(yahoo_iv):
    vol_labels.append("Yahoo Implied Volatility")
    vol_values.append(yahoo_iv)

ax3.bar(vol_labels, vol_values, alpha=0.75, edgecolor="black")

for i, value in enumerate(vol_values):
    ax3.text(i, value, f"{value:.2%}", ha="center", va="bottom", fontsize=11)

```



```

ax3.set_title("Historical Volatility vs Implied Volatility", fontsize=14)
ax3.set_ylabel("Annualized Volatility")
ax3.grid(True, axis="y", alpha=0.2)

# Gráfica 4: precio teórico vs precio de mercado
ax4 = plt.subplot(4, 1, 4)
price_labels = ["Black-Scholes\nHistorical "]
price_values = [bs_price]

if np.isfinite(market_price):
    price_labels.append(f"Market Price\n{market_price_source}")
    price_values.append(market_price)

ax4.bar(price_labels, price_values, alpha=0.75, edgecolor="black")

for i, value in enumerate(price_values):
    ax4.text(i, value, f"${value:.4f}", ha="center", va="bottom", fontsize=11)

ax4.set_title(f"{option_type.capitalize()} Option Price Comparison", fontsize=14)
ax4.set_ylabel("Option Price")
ax4.grid(True, axis="y", alpha=0.2)

plt.tight_layout()
plt.show()

# -----
# 9. Tablas resumen
# -----

summary_data = {
    "Metric": [
        "Ticker",
        "Option Type",
        "Expiration",
        "Current Price S0",
        "Strike K",
        "Maturity T",
        "Risk-Free Rate r",
        "Dividend Yield q",
        "Daily Log Mean",
        "Daily Log Std Dev",
        "Historical Expected Return ",
        "Historical Volatility ",
        "d1",
        "d2",
        "Black-Scholes Price",
    ]
}

```

```

    "Market Price Source",
    "Market Bid",
    "Market Ask",
    "Market Last Price",
    "Market Mid Price",
    "Market Price Used",
    "Implied Volatility from Market Price",
    "Yahoo Implied Volatility",
    "BS - Market Difference",
    "BS - Market Difference (%)"
],
"Value": [
    stock_ticker,
    option_type.capitalize(),
    selected_expiration if selected_expiration is not None else "Manual T",
    f"${S0:.2f}",
    f"${K:.2f}",
    f"${T:.4f} years",
    f"${r:.2%}",
    f"${q:.2%}",
    f"${m:.6f}",
    f"${s:.6f}",
    f"${mu:.2%}",
    f"${sigma:.2%}",
    f"${d1:.4f}",
    f"${d2:.4f}",
    f"${bs_price:.4f}",
    market_price_source,
    f"${market_bid:.4f}" if np.isfinite(market_bid) else "N/A",
    f"${market_ask:.4f}" if np.isfinite(market_ask) else "N/A",
    f"${market_last:.4f}" if np.isfinite(market_last) else "N/A",
    f"${market_mid:.4f}" if np.isfinite(market_mid) else "N/A",
    f"${market_price:.4f}" if np.isfinite(market_price) else "N/A",
    f"${implied_vol_market:.2%}" if np.isfinite(implied_vol_market) else "N/A",
    f"${yahoo_iv:.2%}" if np.isfinite(yahoo_iv) else "N/A",
    f"${pricing_difference:.4f}" if np.isfinite(pricing_difference) else "N/A",
    f"${pricing_difference_pct:.2%}" if np.isfinite(pricing_difference_pct) else "N/A"
]
}

summary_df = pd.DataFrame(summary_data)

greeks_df = pd.DataFrame({
    "Greek": list(greeks.keys()),
    "Value": [f"${value:.6f}" for value in greeks.values()]
})

display(summary_df)

```

```

display(greeks_df)

if market_row is not None:
    market_row_df = pd.DataFrame(market_row).T
    display(market_row_df)

# -----
# 10. Interpretación financiera automática
# -----

print("\nInterpretation:")
print(f"The annualized historical volatility estimated from daily log returns is {sigma:.2%}.")

print(
    f"Using S0 = ${S0:.2f}, K = ${K:.2f}, T = {T:.4f}, r = {r:.2%}, "
    f"and historical = {sigma:.2%}, the Black-Scholes {option_type} price is ${bs_price:.4f}."
)

if np.isfinite(market_price):
    print(
        f"The observed market price used for comparison is ${market_price:.4f} "
        f"based on {market_price_source}. Therefore, BS - Market = ${pricing_difference:.4f}, "
        f"or {pricing_difference_pct:.2%} of the market price."
    )

    if np.isfinite(implied_vol_market):
        print(
            f"The implied volatility backed out from the market price is {implied_vol_market:.2%} "
            f"compared with historical volatility of {sigma:.2%}."
        )

        if implied_vol_market > sigma:
            print(
                "The market-implied volatility is higher than the historical estimate. This usually means "
                "the market is pricing more future uncertainty, volatility risk premium, event risk, "
                "or supply-demand pressure than what appears in the backward-looking historical volatility."
            )

        elif implied_vol_market < sigma:
            print(
                "The market-implied volatility is lower than the historical estimate. This can mean "
                "recent realized volatility was elevated but the market expects calmer future volatility, "
                "or when option liquidity and bid-ask conditions affect the quoted option price."
            )

        else:
            print(

```

```

        "The implied and historical volatilities are very close, so the historical cal.
        "consistent with the listed option price under the Black-Scholes assumptions."
    )

    print(
        "Important caveat: a difference between Black-Scholes and the market price is not automa
        "Black-Scholes assumes constant volatility, lognormal dynamics, frictionless trading, c
        "and a simplified treatment of dividends. Listed option prices also reflect bid-ask sp
        "volatility smiles, jumps, and forward-looking expectations."
    )

else:
    print(
        "No usable market option price was found. The result is therefore a theoretical Black-S
        "calibrated only with historical volatility. To complete the market comparison, use an
        "or last price for the same ticker, expiration, strike, and option type."
    )

print(
    "\nHistorical volatility is backward-looking: it is estimated from past returns. Implied v
    "forward-looking: it is the volatility level that makes Black-Scholes match the observed op
    "For that reason, they can differ even for the same asset and date."
)

```

Untitled5: Parisian down-and-out barrier option

Objective. Price a Parisian down-and-out option and study sensitivity to the required number of consecutive days below a barrier.

Inputs/parameters. User-input ticker with default SPY; ($K=S_0$); ($T=1$); ($r=4.5\%$); barrier ($H=0.95S_0$); option type call; 10,000 paths; daily monitoring over 252 trading days; (D) values of 1, 3, 5, 10, 21, 42, 63, and 126 days.

Method. The code simulates risk-neutral paths, tracks the maximum consecutive number of days each path spends below (H), knocks out paths with max clock ($\geq D$), and discounts surviving payoffs.

Key formulas. Vanilla payoff ($\max(S_T - K, 0)$) for calls or ($\max(K - S_T, 0)$) for puts; Parisian payoff equals zero if the clock condition is triggered.

Code explanation. The code compares the Parisian prices with European Monte Carlo and Black-Scholes benchmarks, plots simulated paths, an example path with its Parisian clock, and price sensitivity to (D).

Output interpretation. The saved AMZN run reports European MC price \$42.0536 and Black-Scholes price \$42.6475. Parisian price increases from \$16.7480 at ($D=1$) to \$41.4630 at ($D=126$), showing that longer memory requirements reduce knock-out probability and increase value.

Assumptions/limitations. Monitoring is daily and discrete. Continuous monitoring or intraday data would change knock-out probability. The barrier is fixed at 95% of (S_0); no rebate is modelled.

Original code cell 0 from Untitled5.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display
from math import erf

# 1. Setup & Data
ticker = input("Enter ticker: ").upper() or "SPY"

df = yf.download(ticker, start="2021-05-01", auto_adjust=True, progress=False)["Close"]
if isinstance(df, pd.DataFrame): df = df.iloc[:, 0]
df = df.dropna()

# 2. Statistics & Calibration
log_returns = np.log(df / df.shift(1)).dropna()

m, s = log_returns.mean(), log_returns.std()
mu = (252 * m) + (0.5 * 252 * s**2)
sigma = np.sqrt(252) * s

# Risk-neutral option parameters
S0 = df.iloc[-1]
K = S0 # At-the-money strike
T = 1.0 # 1 year maturity
r = 0.045 # Annual risk-free rate
H = 0.95 * S0 # Down barrier: 95% of current price
option_type = "call" # Change to "put" if needed

# Monte Carlo parameters
np.random.seed(42)
n_paths = 10000
n_days = int(252 * T)
dt = T / n_days

# Parisian clock values: D in consecutive trading days below H
D_values = np.array([1, 3, 5, 10, 21, 42, 63, 126])
D_values = D_values[D_values <= n_days]

# 3. Vectorized Simulation under Q
Z = np.random.standard_normal((n_days, n_paths))

returns_Q = np.exp(
    (r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z
```

```

)

paths = np.vstack([
    np.full(n_paths, S0),
    S0 * np.cumprod(returns_Q, axis=0)
])

sim_days = np.arange(0, n_days + 1)
ST = paths[-1, :]

# 4. Parisian Barrier Clock
# The clock counts consecutive days below H.
current_clock = np.zeros(n_paths, dtype=int)
max_clock = np.zeros(n_paths, dtype=int)

for t in range(1, n_days + 1):
    below_barrier = paths[t, :] < H
    current_clock = np.where(below_barrier, current_clock + 1, 0)
    max_clock = np.maximum(max_clock, current_clock)

# Plain vanilla payoff before the Parisian knock-out condition
if option_type.lower() == "call":
    vanilla_payoff = np.maximum(ST - K, 0)
elif option_type.lower() == "put":
    vanilla_payoff = np.maximum(K - ST, 0)
else:
    raise ValueError("option_type must be either 'call' or 'put'.")

discount = np.exp(-r * T)

# European Monte Carlo benchmark with the same simulated terminal prices
european_discounted_payoff = discount * vanilla_payoff
european_mc_price = european_discounted_payoff.mean()
european_mc_se = european_discounted_payoff.std(ddof=1) / np.sqrt(n_paths)

# Closed-form Black-Scholes benchmark
def norm_cdf(x):
    return 0.5 * (1 + erf(x / np.sqrt(2)))

def black_scholes_price(S, K, T, r, sigma, option_type="call"):
    if T <= 0:
        return max(S - K, 0) if option_type == "call" else max(K - S, 0)

    if sigma <= 0:
        forward = S * np.exp(r * T)
        payoff = max(forward - K, 0) if option_type == "call" else max(K - forward, 0)
        return np.exp(-r * T) * payoff

```

```

d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
d2 = d1 - sigma * np.sqrt(T)

if option_type.lower() == "call":
    return S * norm_cdf(d1) - K * np.exp(-r * T) * norm_cdf(d2)
else:
    return K * np.exp(-r * T) * norm_cdf(-d2) - S * norm_cdf(-d1)

bs_price = black_scholes_price(S0, K, T, r, sigma, option_type)

# Parisian down-and-out prices for each D
results = []

for D in D_values:
    knocked_out = max_clock >= D
    parisian_payoff = np.where(knocked_out, 0, vanilla_payoff)
    discounted_payoff = discount * parisian_payoff

    price = discounted_payoff.mean()
    std_error = discounted_payoff.std(ddof=1) / np.sqrt(n_paths)

    results.append({
        "D (Trading Days)": D,
        "D (Years)": D / 252,
        "Parisian Price": price,
        "Std Error": std_error,
        "95% CI Low": price - 1.96 * std_error,
        "95% CI High": price + 1.96 * std_error,
        "Survival Rate": 1 - knocked_out.mean(),
        "Knock-Out Rate": knocked_out.mean(),
        "Avg Max Clock": max_clock.mean()
    })

results_df = pd.DataFrame(results)

# 5. Visualization
fig = plt.figure(figsize=(12, 16))

# Chart 1: Daily Log Returns
ax1 = plt.subplot(4, 1, 1)
ax1.plot(log_returns.index, log_returns, color='steelblue', lw=1, alpha=0.7, label='Log Returns')
ax1.axhline(m, color='red', ls='--', label=f'Mean: {m:.4f}')
ax1.axhline(m + s, color='orange', ls=':', label=f'+1 Std Dev: {s:.4f}')
ax1.axhline(m - s, color='orange', ls=':')
ax1.set_title(f"{ticker} Análisis de Retornos Logarítmicos", fontsize=14)
ax1.legend(loc='upper left')
ax1.grid(True, alpha=0.2)

```

Chart 2: Simulated Risk-Neutral Paths

```
ax2 = plt.subplot(4, 1, 2)

sample_size = min(80, n_paths)
sample_idx = np.random.choice(n_paths, size=sample_size, replace=False)

ax2.plot(sim_days, paths[:, sample_idx], color='steelblue', alpha=0.18, lw=0.8)
ax2.plot(sim_days, paths.mean(axis=1), color='red', lw=2, label='Mean Simulated Path')
ax2.axhline(K, color='black', ls='--', lw=1.5, label=f"Strike K = {K:.2f}")
ax2.axhline(H, color='darkred', ls='--', lw=1.5, label=f"Barrier H = {H:.2f}")
ax2.set_title(f"{ticker} Trayectorias Simuladas bajo Q con Barrera Parisina", fontsize=14)
ax2.set_ylabel("Price ($)")
ax2.legend(loc='upper left')
ax2.grid(True, alpha=0.2)
```

Chart 3: One Path and Its Parisian Clock

```
D_example = int(21 if 21 in D_values else D_values[len(D_values) // 2])
```

```
example_candidates = np.where(max_clock >= D_example)[0]
if len(example_candidates) > 0:
    example_idx = example_candidates[0]
else:
    example_idx = np.argmax(max_clock)
```

```
example_path = paths[:, example_idx]
```

```
example_clock = np.zeros(n_days + 1, dtype=int)
for t in range(1, n_days + 1):
    if example_path[t] < H:
        example_clock[t] = example_clock[t - 1] + 1
    else:
        example_clock[t] = 0
```

```
ax3 = plt.subplot(4, 1, 3)
ax3.plot(sim_days, example_path, color='steelblue', lw=1.8, label='Example Path')
ax3.axhline(H, color='darkred', ls='--', lw=1.5, label=f"Barrier H = {H:.2f}")
ax3.axhline(K, color='black', ls=':', lw=1.2, label=f"Strike K = {K:.2f}")
ax3.set_title(f"Reloj Parisino Consecutivo bajo H | D = {D_example} días", fontsize=14)
ax3.set_ylabel("Price ($)")
ax3.legend(loc='upper left')
ax3.grid(True, alpha=0.2)
```

```
ax3b = ax3.twinx()
ax3b.step(sim_days, example_clock, color='purple', lw=1.8, alpha=0.8, label='Consecutive Clock')
ax3b.axhline(D_example, color='purple', ls='--', lw=1.2, alpha=0.7, label=f"D = {D_example}")
ax3b.set_ylabel("Consecutive Days Below H")
```

Chart 4: Sensitivity to Parisian Clock D


```

ax4 = plt.subplot(4, 1, 4)
ax4.errorbar(
    results_df["D (Trading Days)"],
    results_df["Parisian Price"],
    yerr=1.96 * results_df["Std Error"],
    fmt='o-',
    capsize=4,
    color='steelblue',
    label='Parisian Down-and-Out Price'
)

ax4.axhline(european_mc_price, color='red', ls='--', lw=1.8, label='European MC Benchmark')
ax4.axhline(bs_price, color='black', ls=':', lw=1.8, label='Black-Scholes Benchmark')
ax4.set_title(f"Sensibilidad del Precio Parisino al Reloj D", fontsize=14)
ax4.set_xlabel("D: Consecutive Trading Days Below Barrier")
ax4.set_ylabel("Option Price ($)")
ax4.legend(loc='lower right')
ax4.grid(True, alpha=0.2)

plt.tight_layout()
plt.show()

# 6. Summary Tables
summary_data = {
    "Metric": [
        "Ticker",
        "Current Price S0",
        "Strike K",
        "Barrier H",
        "Barrier as % of S0",
        "Maturity T",
        "Risk-Free Rate r",
        "Daily Log Mean",
        "Daily Log Std Dev",
        "Yearly Expected Return ()",
        "Yearly Volatility ()",
        "Number of Paths",
        "Trading Days Simulated",
        "European MC Price",
        "European MC Std Error",
        "Black-Scholes Price"
    ],
    "Value": [
        ticker,
        f"${S0:.2f}",
        f"${K:.2f}",
        f"${H:.2f}",
        f"${H / S0:.2%}",
    ]
}

```

```

        f"{T:.2f} years",
        f"{r:.2%}",
        f"{m:.6f}",
        f"{s:.6f}",
        f"{mu:.2%}",
        f"{sigma:.2%}",
        f"{n_paths:,}",
        f"{n_days}",
        f"${european_mc_price:.4f}",
        f"${european_mc_se:.4f}",
        f"${bs_price:.4f}"
    ]
}

```

```
summary_df = pd.DataFrame(summary_data)
```

```
display(summary_df)
display(results_df.round(6))
```

7. Financial Interpretation

```

lowest_D_price = results_df.iloc[0]["Parisian Price"]
highest_D_price = results_df.iloc[-1]["Parisian Price"]
price_change = highest_D_price - lowest_D_price

```

```

print(f"""
Interpretación financiera:

```

La opción simulada es una `{option_type.upper()}` down-and-out parisina. Esto significa que no se desactiva por tocar la barrera H una sola vez, sino por permanecer por debajo de H durante D días consecutivos.

Cuando D aumenta, el knock-out se vuelve más difícil de activar. Por eso, en general, el precio parisino aumenta conforme D crece. En esta simulación, el precio pasa de aproximadamente `${lowest_D_price:.4f}` con `D = {int(results_df.iloc[0]["D (Trading Days)"])} días` a `${highest_D_price:.4f}` con `D = {int(results_df.iloc[-1]["D (Trading Days)"])} días`.

La diferencia de `${price_change:.4f}` representa el valor económico de la memoria temporal: la opción filtra cruces breves por debajo de la barrera y sólo castiga trayectorias que permanecen en zona de estrés durante varios días consecutivos.

Nota técnica: esta implementación usa monitoreo diario discreto. Con monitoreo continuo, el precio puede cambiar porque algunos cruces intradía no observados por datos diarios sí podrían activar o acelerar el reloj parisino.

```

""")

```

Untitled3: GARCH(1,1), conditional volatility, RSI, and automated interpretation

Objective. Analyze a stock's price, returns, volume, RSI, and conditional volatility using GARCH(1,1).

Inputs/parameters. User-input ticker; Yahoo Finance prices from 2021-05-01 in source code; GARCH(1,1) with normal distribution; returns rescaled by multiplying by 1000.

Method. The code computes log returns, volume changes, RSI using exponential moving averages of gains/losses, fits `arch.arch_model(..., vol='Garch', p=1, q=1)`, extracts conditional volatility, and builds Altair charts.

Key formulas. $(RSI=100-\frac{100}{1+RS})$, with $(RS=\frac{\text{avg gain}}{\text{avg loss}})$.
GARCH variance: $(\sigma_t^2=\omega+\alpha_1\epsilon_{t-1}^2+\beta_1\sigma_{t-1}^2)$.

Code explanation. The code installs `arch`, fits the model, prints the model summary, creates interactive charts, then constructs an automated interpretation using estimated parameters and p-values.

Output interpretation. The saved NFLX run reports $(\alpha_1=0.0174)$, $(\beta_1=0.9751)$, and $(\alpha+\beta\approx0.9925)$. This indicates high volatility persistence. (α_1) and (β_1) are significant in the saved output; (ω) is not significant.

Assumptions/limitations. The output warning references a start date inconsistent with the visible source, suggesting stale output metadata. Rescaling by 1000 affects parameter scale. GARCH models volatility, not expected price direction.

Original code cell 0 from Untitled3.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import yfinance as yf
import pandas as pd
import numpy as np
import altair as alt
import arch

# --- Part 1: Stock Selection ---
# This section handles the selection of the stock ticker and downloads its historical data.

ticker = input("Enter the stock ticker symbol: ")
Prices = yf.download(ticker, start="2021-05-01")
ticker_prices = Prices["Close"]
```

Original code cell 1 from Untitled3.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import altair as alt
import arch
import numpy as np # Ensure numpy is explicitly imported for clarity
```

```

# --- Part 2: Stock Analysis (Graphs and GARCH Model) ---
# This section performs various analyses on the selected stock, including interactive charts and

# --- Prepare data for charts and GARCH model ---
prices_df = ticker_prices.reset_index()
prices_df.columns = ['Date', 'Close']

daily_returns = np.log(Prices['Close'] / Prices['Close'].shift(1))
returns_df = daily_returns.reset_index()
returns_df.columns = ['Date', 'Daily Log Return']

stock_volume = Prices['Volume']
volume_df = stock_volume.reset_index()
volume_df.columns = ['Date', 'Volume']

volume_change = Prices['Volume'].diff()
volume_change_df = volume_change.reset_index()
volume_change_df.columns = ['Date', 'Volume Change']

# --- Calculate Relative Strength Index (RSI) ---
rsi_period = 14 # Common RSI period
delta = Prices['Close'].diff(1)
gain = delta.clip(lower=0) # Positive changes
loss = -delta.clip(upper=0) # Negative changes (as positive values)

# Calculate Exponentially Weighted Moving Average (EWMA) for gains and losses
avg_gain = gain.ewm(com=rsi_period - 1, adjust=False).mean()
avg_loss = loss.ewm(com=rsi_period - 1, adjust=False).mean()

# Calculate Relative Strength (RS) and RSI
rs = avg_gain / avg_loss
rsi = 100 - (100 / (1 + rs))

rsi_df = rsi.reset_index()
rsi_df.columns = ['Date', 'RSI']

# --- GARCH Model for Volatility Analysis (Moved up to generate conditional volatility for charts) ---
# Install arch library if not already installed (this line will execute every time the cell runs)
!pip install arch

# Drop any NaN values from daily_returns before fitting the model
# daily_returns is already defined from the Prices dataframe above.
returns_for_garch = daily_returns.dropna() * 1000 # Rescale returns to avoid DataScaleWarning

# Define the GARCH(1,1) model

```

```

# The mean model is specified as ConstantMean by default in arch.arch_model
# and the distribution is assumed to be Normal by default.
# Using p=1, q=1 for GARCH(1,1)
garch_model = arch.arch_model(returns_for_garch, vol='Garch', p=1, q=1, dist='normal')

# Fit the GARCH model
garch_results = garch_model.fit(disps='off')

# Print the model summary
print(garch_results.summary())

# --- Extract Conditional Volatility for Charting ---
conditional_volatility = garch_results.conditional_volatility
# The index of conditional_volatility corresponds to the dates of returns_for_garch
volatility_df = conditional_volatility.reset_index()
volatility_df.columns = ['Date', 'Conditional Volatility']

# --- Interactive Charts (re-ordered to after GARCH fitting for volatility_chart) ---

# 1. Stock Price Over Time
chart_price = alt.Chart(prices_df).mark_line().encode(
    x=alt.X('Date:T', title='Date'),
    y=alt.Y('Close:Q', title='Close Price'),
    tooltip=['Date', 'Close']
).properties(
    title=f'{ticker} Stock Price Over Time'
).interactive()

# 2. Daily Log Return Over Time
returns_chart = alt.Chart(returns_df).mark_line().encode(
    x=alt.X('Date:T', title='Date'),
    y=alt.Y('Daily Log Return:Q', title='Daily Log Return'),
    tooltip=['Date', 'Daily Log Return']
).properties(
    title=f'{ticker} Daily Log Return Over Time'
).interactive()

# 3. Trading Volume Over Time
volume_chart = alt.Chart(volume_df).mark_area(opacity=0.7).encode(
    x=alt.X('Date:T', title='Date'),
    y=alt.Y('Volume:Q', title='Trading Volume'),
    tooltip=['Date', 'Volume']
).properties(
    title=f'{ticker} Trading Volume Over Time'
).interactive()

```

```

# 4. Daily Volume Change Over Time
volume_change_chart = alt.Chart(volume_change_df).mark_line().encode(
    x=alt.X('Date:T', title='Date'),
    y=alt.Y('Volume Change:Q', title='Daily Volume Change'),
    tooltip=['Date', 'Volume Change']
).properties(
    title=f'{ticker} Daily Volume Change Over Time'
).interactive()

# 5. Conditional Volatility Over Time
volatility_chart = alt.Chart(volatility_df).mark_line().encode(
    x=alt.X('Date:T', title='Date'),
    y=alt.Y('Conditional Volatility:Q', title='Conditional Volatility'),
    tooltip=['Date', 'Conditional Volatility']
).properties(
    title=f'{ticker} Conditional Volatility (GARCH) Over Time'
).interactive()

# 6. Relative Strength Index (RSI) Over Time
rsi_chart = alt.Chart(rsi_df).mark_line().encode(
    x=alt.X('Date:T', title='Date'),
    y=alt.Y('RSI:Q', title='RSI'),
    tooltip=['Date', 'RSI']
).properties(
    title=f'{ticker} Relative Strength Index (RSI) Over Time ({rsi_period}-day)'
).interactive()

# Combine all charts into a single vertical concatenation
combined_chart = alt.vconcat(
    chart_price,
    returns_chart,
    volume_chart,
    volume_change_chart,
    volatility_chart, # Add new volatility chart
    rsi_chart        # Add new RSI chart
)
display(combined_chart)

```

Original code cell 2 from Untitled3.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

--- Part 3: Automated Time Series Interpretation ---

```

# 1. Descriptive Statistics of Daily Returns
# Note: daily_returns are now log returns (decimals)
mean_return = daily_returns.mean().iloc[0] * 100 # Convert to percentage for display
std_dev_return = daily_returns.std().iloc[0] * 100 # Convert to percentage for display

```

```

skewness_return = daily_returns.skew().iloc[0]
kurtosis_return = daily_returns.kurt().iloc[0]

# 2. GARCH Model Interpretation
garch_params = garch_results.params
omega = garch_params.loc['omega']
alpha1 = garch_params.loc['alpha[1]']
beta1 = garch_params.loc['beta[1]']

garch_pvalues = garch_results.pvalues
p_mu = garch_pvalues.loc['mu']
p_omega = garch_pvalues.loc['omega']
p_alpha1 = garch_pvalues.loc['alpha[1]']
p_beta1 = garch_pvalues.loc['beta[1]']

# Constructing the interpretation string
interpretation_text = f"## Automated Time Series Interpretation for {ticker}\n\n"

interpretation_text += "### 1. Daily Log Returns Characteristics:\n"
interpretation_text += f"- **Mean Daily Log Return:** {mean_return:.4f}%\n"
interpretation_text += f"- **Standard Deviation (Volatility):** {std_dev_return:.4f}%\n"
interpretation_text += f"- **Skewness:** {skewness_return:.4f} (A negative value suggests more\n"
interpretation_text += f"- **Kurtosis:** {kurtosis_return:.4f} (A value > 3 suggests 'fat tails\n"

interpretation_text += "These statistics indicate the central tendency, dispersion, and shape of\n"

interpretation_text += "### 2. GARCH(1,1) Volatility Analysis:\n"
interpretation_text += f"The GARCH(1,1) model was fitted to analyze the stock's conditional volatility.\n"
interpretation_text += f"- **omega (Constant Term):** {omega:.4f} (p-value: {p_omega:.4f})\n"
interpretation_text += f"- **alpha[1] (ARCH Term - Impact of Shocks):** {alpha1:.4f} (p-value: {p_alpha1:.4f})\n"
interpretation_text += f"- **beta[1] (GARCH Term - Volatility Persistence):** {beta1:.4f} (p-value: {p_beta1:.4f})\n"

interpretation_text += "Based on these parameters:\n"
if p_alpha1 < 0.05:
    interpretation_text += f"- The **alpha[1]** coefficient is statistically significant (p < 0.05).\n"
else:
    interpretation_text += f"- The **alpha[1]** coefficient is not statistically significant (p > 0.05).\n"

if p_beta1 < 0.05:
    interpretation_text += f"- The **beta[1]** coefficient is statistically significant (p < 0.05).\n"
else:
    interpretation_text += f"- The **beta[1]** coefficient is not statistically significant (p > 0.05).\n"

if p_omega < 0.05:
    interpretation_text += f"- The **omega** term is statistically significant, representing the baseline volatility.\n"
else:
    interpretation_text += f"- The **omega** term is not statistically significant, suggesting the volatility is primarily driven by shocks."

```

```

alpha_plus_beta = alpha1 + beta1
interpretation_text += f"- The sum of **alpha[1] + beta[1]** is {alpha_plus_beta:.4f}. A sum c

interpretation_text += "Overall, the GARCH model highlights the presence of volatility cluster

interpretation_text += "### 3. Forecasting Capability:\n"
interpretation_text += "- **Price Forecasting:** Forecasting the *direction* of stock prices i
interpretation_text += "- **Volatility Forecasting:** The GARCH model is primarily designed fo
interpretation_text += "In conclusion, while direct stock price forecasting remains a signific

```

Original code cell 3 from Untitled3.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

from IPython.display import Markdown, display

display(Markdown(interpretation_text))

```

Original code cell 5 from Untitled3.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

Series de Tiempo: AR, ARIMA, Prophet, SARIMA, AIC/BIC, and diagnostics

Objective. Demonstrate time-series simulation, model comparison, diagnostics, and forecasting with AR/ARIMA, Prophet, and SARIMA.

Inputs/parameters. Simulated AR(1) series with ($\phi=0.5$), ($n=200$), ($\sigma=1$); Melbourne minimum-temperature dataset from GitHub; SARIMA seasonal period ($s=7$); ARIMA and SARIMA grids for AIC/BIC.

Method. The notebook visualizes AR(1), compares AR(1) and AR(2), plots residuals and Q-Q diagnostics, computes ACF/PACF, searches ARIMA orders, fits Prophet, fits SARIMA, and searches seasonal model orders.

Key formulas. ($X_t = c + \phi X_{t-1} + \epsilon_t$), $\text{ARIMA}((p,d,q))$, $\text{SARIMA}((p,d,q)(P,D,Q)_s)$, ($\text{AIC} = 2k - 2 \ln L$), and ($\text{BIC} = k \ln n - 2 \ln L$).

Code explanation. The original notebook is stateful: early cells visualize variables before they are defined later. The consolidated version preserves the code order and documents these dependencies rather than correcting them.

Output interpretation. Saved AR example: AR(1) had lower AIC/BIC than AR(2). Later grid search selected ARIMA(2,0,1) by AIC and ARIMA(1,0,0) by BIC. Prophet fitted the Melbourne temperature data and generated 365-day forecasts. SARIMA search selected SARIMA(1,1,1)(0,1,1,7) by both AIC and BIC.

Assumptions/limitations. Fresh top-to-bottom execution will fail unless prerequisite variables are created before early visualization cells. Cell 28 has a saved `NameError` because `model_orders`

was not defined before use. Cell 55 requires a local `california_housing_train.csv` not attached. Cell 58 downloads ticker BK but plots `y=PLTR`, which is likely inconsistent.

Original code cell 0 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import pandas as pd
import yfinance as yf
```

Original markdown cell 1 from Series de Tiempo.ipynb

Visualización de la Serie Temporal Generada

Primero, vamos a recordar cómo se ve la serie `ar1_series` que hemos simulado. Esto nos dará una base visual para entender los ajustes de los modelos.

Original code cell 2 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
plt.plot(ar1_series)
plt.title('Serie Temporal Simulada AR(1)')
plt.xlabel('Paso de Tiempo')
plt.ylabel('Valor')
plt.grid(True)
plt.show()
```

Original markdown cell 3 from Series de Tiempo.ipynb

Comparación Visual de los Ajustes del Modelo

Ahora, vamos a superponer los valores ajustados por los modelos AR(1) y AR(2) sobre la serie original. Esto nos permitirá ver visualmente qué tan bien cada modelo replica el patrón de los datos.

Original code cell 4 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
plt.figure(figsize=(14, 7))
plt.plot(ar1_series, label='Serie Original', color='blue')
plt.plot(results_ar1.predict(start=0, end=len(ar1_series)-1), label='Ajuste AR(1)', color='red')
plt.plot(results_ar2.predict(start=0, end=len(ar1_series)-1), label='Ajuste AR(2)', color='green')
plt.title('Serie Original vs. Ajustes de Modelos AR(1) y AR(2)')
plt.xlabel('Paso de Tiempo')
plt.ylabel('Valor')
```

```
plt.legend()
plt.grid(True)
plt.show()
```

Original markdown cell 5 from `Series de Tiempo.ipynb`

Análisis de los Residuos del Modelo

Los residuos son la diferencia entre los valores observados y los valores predichos por el modelo. Un buen modelo debería tener residuos que se asemejen a ruido blanco (sin patrones). Graficar los residuos nos ayuda a evaluar la calidad del ajuste y a entender la 'información perdida' que AIC y BIC intentan cuantificar.

Original code cell 6 from `Series de Tiempo.ipynb` Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)
```

```
# Residuos del Modelo AR(1)
axes[0].plot(results_ar1.resid, color='red')
axes[0].set_title('Residuos del Modelo AR(1)')
axes[0].set_ylabel('Residuo')
axes[0].grid(True)
```

```
# Residuos del Modelo AR(2)
axes[1].plot(results_ar2.resid, color='green')
axes[1].set_title('Residuos del Modelo AR(2)')
axes[1].set_xlabel('Paso de Tiempo')
axes[1].set_ylabel('Residuo')
axes[1].grid(True)
```

```
plt.tight_layout()
plt.show()
```

Original markdown cell 7 from `Series de Tiempo.ipynb`

Interpretación de las Gráficas y su Relación con AIC/BIC

- **Gráfica de Serie Temporal:** Muestra los datos que estamos intentando modelar.
- **Comparación de Ajustes:** Idealmente, la línea de ajuste de un buen modelo debería seguir de cerca la serie original. Observaremos que el ajuste del AR(1) será muy similar al AR(2) visualmente, ya que la serie fue generada por un proceso AR(1).
- **Gráfica de Residuos:** Los residuos del modelo AR(1) deberían ser más cercanos al ruido blanco, lo que indica que el modelo ha capturado la mayor parte de la estructura en los datos. Los residuos del AR(2) podrían mostrar un patrón similar pero no necesariamente mejor, ya que el parámetro adicional no es fundamental para la serie generada. Los valores de AIC y BIC formalizan esta observación visual, penalizando la complejidad adicional del AR(2) si no aporta una mejora significativa en el ajuste (es decir, en la reducción del ruido en los residuos).

Original markdown cell 8 from Series de Tiempo.ipynb

Criterio de Información de Akaike (AIC)

El **Criterio de Información de Akaike (AIC)** es una medida de la calidad relativa de los modelos estadísticos para un conjunto de datos dado. Como tal, el AIC proporciona un medio para la selección del modelo. Dado un conjunto de modelos candidatos para los datos, el modelo preferido es aquel con el valor AIC más bajo.

El AIC se basa en la teoría de la información y estima la información perdida cuando se utiliza un modelo para representar el proceso que generó los datos. Se calcula como:

$$\text{AIC} = 2k - 2\ln(L)$$

Donde:

- k es el número de parámetros en el modelo.
- L es el valor máximo de la función de verosimilitud (likelihood) del modelo.

Un valor de AIC más bajo indica un mejor ajuste del modelo, teniendo en cuenta la simplicidad del mismo. Penaliza la adición de parámetros (complejidad) para evitar el sobreajuste.

Original markdown cell 9 from Series de Tiempo.ipynb

Criterio de Información Bayesiano (BIC)

El **Criterio de Información Bayesiano (BIC)**, también conocido como Criterio de Schwarz (SIC), es otro criterio para la selección de modelos entre un conjunto finito de modelos. Similar al AIC, busca el modelo con el valor BIC más bajo.

El BIC introduce una penalización más fuerte para el número de parámetros en el modelo que el AIC. Se calcula como:

$$\text{BIC} = k\ln(n) - 2\ln(L)$$

Donde:

- k es el número de parámetros en el modelo.
- n es el número de observaciones (tamaño de la muestra).
- L es el valor máximo de la función de verosimilitud del modelo.

Debido a la penalización $k\ln(n)$, que es más grande que $2k$ (especialmente para n grandes), el BIC tiende a seleccionar modelos más parsimoniosos (con menos parámetros) que el AIC. Es útil cuando se busca un modelo que sea una buena representación del proceso de generación de datos, incluso si significa un ligero compromiso en el ajuste a los datos observados.

Original markdown cell 10 from Series de Tiempo.ipynb

Ejemplo: Comparación de Modelos AR con AIC y BIC

Usaremos la serie temporal `ar1_series` que ya generamos, y ajustaremos un modelo autorregresivo (AR) de orden 1 y otro de orden 2 para comparar sus valores de AIC y BIC. El modelo con el AIC o BIC más bajo será considerado el "mejor" para nuestros datos.

Original code cell 11 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

Original code cell 12 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA
import numpy as np # Se añade import numpy para la generación de ar1_series

# --- Generar ar1_series --- (Copiar el código de la celda 64026f11 para asegurar su definición)
# Parameters for the AR(1) process
phi = 0.5 # Autoregressive coefficient (must be < 1 for stationarity)
c = 0     # Constant term
sigma = 1 # Standard deviation of the white noise
n = 200   # Number of observations

# Initialize the time series array
ar1_series = np.zeros(n)

# Generate white noise (innovations)
white_noise = np.random.normal(0, sigma, n)

# Simulate the AR(1) process
#  $X_t = c + \phi * X_{t-1} + \epsilon_t$ 
for t in range(1, n):
    ar1_series[t] = c + phi * ar1_series[t-1] + white_noise[t]
# --- Fin de la generación de ar1_series ---

# --- Ajustar un modelo AR(1) ---
# El orden (p, d, q) para AR(p) es (p, 0, 0)
model_ar1 = ARIMA(ar1_series, order=(1, 0, 0))
results_ar1 = model_ar1.fit()

print("\n--- Resultados del Modelo AR(1) ---")
print(f"AIC: {results_ar1.aic:.2f}")
print(f"BIC: {results_ar1.bic:.2f}")

# --- Ajustar un modelo AR(2) ---
model_ar2 = ARIMA(ar1_series, order=(2, 0, 0))
results_ar2 = model_ar2.fit()

print("\n--- Resultados del Modelo AR(2) ---")
```

```

print(f"AIC: {results_ar2.aic:.2f}")
print(f"BIC: {results_ar2.bic:.2f}")

# --- Comparación ---
print("\n--- Comparación AIC/BIC ---")
if results_ar1.aic < results_ar2.aic:
    print(f"El modelo AR(1) tiene un AIC más bajo ({results_ar1.aic:.2f}) que el AR(2) ({results_ar2.aic:.2f})")
else:
    print(f"El modelo AR(2) tiene un AIC más bajo ({results_ar2.aic:.2f}) que el AR(1) ({results_ar1.aic:.2f})")

if results_ar1.bic < results_ar2.bic:
    print(f"El modelo AR(1) tiene un BIC más bajo ({results_ar1.bic:.2f}) que el AR(2) ({results_ar2.bic:.2f})")
else:
    print(f"El modelo AR(2) tiene un BIC más bajo ({results_ar2.bic:.2f}) que el AR(1) ({results_ar1.bic:.2f})")

```

Original markdown cell 13 from `Series de Tiempo.ipynb`

Interpretación del Ejemplo

En este ejemplo, dado que la serie fue generada como un proceso AR(1) puro, esperamos que el modelo AR(1) tenga valores de AIC y BIC más bajos que el modelo AR(2). El AR(2) introduce un parámetro adicional que no mejora significativamente el ajuste del modelo, pero sí aumenta la penalización por complejidad, haciendo que sus valores de AIC y BIC sean probablemente más altos.

- **AIC** selecciona el modelo que minimiza la pérdida de información.
- **BIC** selecciona el modelo que tiene la mayor probabilidad posterior, suponiendo que el verdadero modelo está en el conjunto de modelos candidatos. Tiende a favorecer modelos más simples que el AIC, especialmente con grandes tamaños de muestra.

Original code cell 14 from `Series de Tiempo.ipynb` Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

import numpy as np
import matplotlib.pyplot as plt

# Parameters for the AR(1) process
phi = 0.5 # Autoregressive coefficient (must be < 1 for stationarity)
c = 0     # Constant term
sigma = 1 # Standard deviation of the white noise
n = 200   # Number of observations

# Initialize the time series array
ar1_series = np.zeros(n)

# Generate white noise (innovations)
white_noise = np.random.normal(0, sigma, n)

```

```

# Simulate the AR(1) process
#  $X_t = c + \phi * X_{t-1} + \epsilon_t$ 
for t in range(1, n):
    ar1_series[t] = c + phi * ar1_series[t-1] + white_noise[t]

# Plot the simulated time series
plt.figure(figsize=(12, 6))
plt.plot(ar1_series)
plt.title(f'Simulated Stationary AR(1) Time Series (phi={phi})')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.grid(True)
plt.show()

```

Original markdown cell 15 from Series de Tiempo.ipynb

Modelos ARIMA (AutoRegressive Integrated Moving Average)

Un modelo ARIMA es una generalización de los modelos de promedios móviles (MA) y autorregresivos (AR). Son ampliamente utilizados para la modelización de series temporales. La "I" en ARIMA significa "Integrado", lo que se refiere al uso de la diferenciación para hacer que la serie temporal sea estacionaria.

Un modelo ARIMA se especifica por 3 órdenes: $\text{ARIMA}(p, d, q)$:

- **p (orden autorregresivo):** El número de observaciones pasadas que se incluirán en el modelo. Representa el número de términos AR.
- **d (orden de diferenciación):** El número de veces que las observaciones brutas son diferenciadas para hacer que la serie temporal sea estacionaria. La diferenciación implica restar la observación actual de una observación pasada (por ejemplo, la observación anterior) para eliminar tendencias o estacionalidad.
- **q (orden de media móvil):** El número de errores de predicción pasados que se incluirán en el modelo. Representa el número de términos MA.

Estacionariedad: Una serie temporal es estacionaria si sus propiedades estadísticas (media, varianza, autocorrelación) no cambian con el tiempo. Los modelos ARIMA asumen que la serie es estacionaria. Si la serie no es estacionaria, se aplica la diferenciación ($d > 0$) para transformarla en una serie estacionaria.

Original markdown cell 16 from Series de Tiempo.ipynb

Funciones de Autocorrelación (ACF) y Autocorrelación Parcial (PACF)

Las gráficas de ACF (Autocorrelation Function) y PACF (Partial Autocorrelation Function) son herramientas esenciales en el análisis de series temporales para identificar el orden apropiado de los componentes AR y MA en un modelo ARIMA.

Función de Autocorrelación (ACF)

La **ACF** mide la correlación entre una serie temporal y sus versiones rezagadas en diferentes intervalos de tiempo. En una gráfica de ACF, el eje x representa el número de rezagos (lags) y el eje y representa el coeficiente de autocorrelación.

- **AR(p) Model:** La ACF decae exponencialmente o con un patrón sinusoidal.
- **MA(q) Model:** La ACF se corta abruptamente (cae a cero) después de q rezagos.

Función de Autocorrelación Parcial (PACF)

La **PACF** mide la correlación entre una serie temporal y sus versiones rezagadas, pero eliminando la influencia de las correlaciones de los rezagos intermedios. Es decir, mide la correlación directa entre una observación y una observación rezagada.

- **AR(p) Model:** La PACF se corta abruptamente (cae a cero) después de p rezagos.
- **MA(q) Model:** La PACF decae exponencialmente o con un patrón sinusoidal.

Original markdown cell 17 from Series de Tiempo.ipynb

Cálculo y Gráficas de ACF y PACF

Vamos a calcular y graficar la ACF y PACF para nuestra serie `ar1_series`. Esto nos ayudará a identificar los posibles órdenes p y q para un modelo ARIMA.

Original code cell 18 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt

plt.figure(figsize=(15, 6))

plt.subplot(121) # 1 fila, 2 columnas, primer gráfico
plot_acf(ar1_series, lags=30, ax=plt.gca(), title='Autocorrelación (ACF)')
plt.xlabel('Rezagos')
plt.ylabel('Autocorrelación')
plt.grid(True)
# Añadir marcador para Q (esperamos decaimiento gradual para AR(1), no un corte para Q)
# En un AR(1), la ACF decae exponencialmente, no se "corta" para un q.
# Podríamos marcar donde se vuelve no significativa o simplemente observar el decaimiento.
# Para fines ilustrativos de donde buscar un 'q' si fuera un MA, no hay un 'q' obvio aquí.
# Dejaremos la interpretación textual de la ACF para AR(1).

plt.subplot(122) # 1 fila, 2 columnas, segundo gráfico
plot_pacf(ar1_series, lags=30, ax=plt.gca(), title='Autocorrelación Parcial (PACF)')
plt.xlabel('Rezagos')
plt.ylabel('Autocorrelación Parcial')
plt.grid(True)
```

```
# Añadir marcador para P (esperamos un corte en el rezago 1 para AR(1))
plt.axvline(x=1, color='red', linestyle='--', label='P = 1 (corte significativo)')
plt.legend()

plt.tight_layout()
plt.show()
```

Original markdown cell 19 from `Series de Tiempo.ipynb`

Interpretación de las Gráficas ACF y PACF

- **En la gráfica de PACF:** Observa dónde la función de autocorrelación parcial se corta abruptamente (cae a cero) o se vuelve no significativa (dentro de las bandas de confianza). El número de rezagos antes de este corte sugiere el orden p del componente AR.
 - Para una serie AR(1) como la `ar1_series` que generamos, esperamos que la PACF tenga un pico significativo en el rezago 1 y luego caiga a cero rápidamente.
- **En la gráfica de ACF:** Observa dónde la función de autocorrelación se corta abruptamente. Este número de rezagos sugiere el orden q del componente MA.
 - Para una serie AR(1) pura, la ACF debería decaer gradualmente (exponencialmente o con un patrón sinusoidal), no cortarse abruptamente.
- **Para la diferenciación (d):** Si la ACF decae muy lentamente y es muy persistente en valores altos, podría indicar que la serie no es estacionaria y que se necesita diferenciación ($d > 0$). En nuestro caso, como la serie es generada AR(1) estacionaria, esperamos $d=0$.

Original markdown cell 20 from `Series de Tiempo.ipynb`

Ajuste de Modelos ARIMA y Comparación de AIC/BIC

Ahora vamos a ajustar diferentes modelos ARIMA (iterando sobre posibles órdenes p y q y manteniendo $d=0$ ya que nuestra serie es estacionaria) y calcularemos sus valores de AIC y BIC. Luego, graficaremos estos valores para visualizar qué combinaciones de órdenes (p , q) resultan en los modelos más adecuados según estos criterios.

Original code cell 21 from `Series de Tiempo.ipynb` Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

Original markdown cell 22 from `Series de Tiempo.ipynb`

Revisión Visual de los Residuos de los Modelos Óptimos

Vamos a graficar los residuos de los dos modelos que fueron seleccionados como 'óptimos' según los criterios AIC y BIC. Como recordatorio:

- El modelo **ARIMA(1,0,0)** fue el 'mejor' según **BIC**.

- El modelo **ARIMA(1,0,1)** fue el 'mejor' según **AIC**.

Idealmente, los residuos de un buen modelo deben parecerse a un ruido blanco: distribuidos aleatoriamente alrededor de cero, sin patrones evidentes, autocorrelación significativa o cambios en la varianza.

Original code cell 23 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA

# results_ar1 (ARIMA(1,0,0)) ya está disponible de celdas anteriores

# Ajustar el modelo ARIMA(1,0,1) si no se ha hecho explícitamente y sus resultados no están guardados
try:
    # Intentar usar results_best_aic si ya se definió en algún punto
    _ = results_best_aic.aic # Check if it exists
except NameError:
    # Si no existe, ajustarlo ahora
    model_best_aic = ARIMA(ar1_series, order=(1, 0, 1))
    results_best_aic = model_best_aic.fit()

fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)

# Residuos del Modelo ARIMA(1,0,0) (Mejor según BIC)
axes[0].plot(results_ar1.resid, color='blue')
axes[0].set_title('Residuos del Modelo ARIMA(1,0,0) (Mejor según BIC)')
axes[0].set_ylabel('Residuo')
axes[0].grid(True)
axes[0].axhline(0, color='black', linestyle='--')

# Residuos del Modelo ARIMA(1,0,1) (Mejor según AIC)
axes[1].plot(results_best_aic.resid, color='green')
axes[1].set_title('Residuos del Modelo ARIMA(1,0,1) (Mejor según AIC)')
axes[1].set_xlabel('Paso de Tiempo')
axes[1].set_ylabel('Residuo')
axes[1].grid(True)
axes[1].axhline(0, color='black', linestyle='--')

plt.tight_layout()
plt.show()
```

Original markdown cell 24 from Series de Tiempo.ipynb

Gráficos Q-Q de los Residuos del Modelo

Los gráficos Q-Q son una herramienta visual para evaluar si una muestra de datos (en este caso, nuestros residuos) sigue una distribución particular (generalmente la distribución normal). Si los puntos en el gráfico Q-Q se alinean aproximadamente a lo largo de una línea recta de 45 grados, sugiere que los residuos están distribuidos normalmente. Cualquier desviación de esta línea indica una desviación de la normalidad.

Original code cell 25 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import statsmodels.graphics.gofplots as smg
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 1, figsize=(12, 10))

# Q-Q Plot para los Residuos del Modelo ARIMA(1,0,0) (Mejor según BIC)
smg.qqplot(results_ar1.resid, line='s', ax=axes[0])
axes[0].set_title('Q-Q Plot de Residuos ARIMA(1,0,0)')

# Q-Q Plot para los Residuos del Modelo ARIMA(1,0,1) (Mejor según AIC)
smg.qqplot(results_best_aic.resid, line='s', ax=axes[1])
axes[1].set_title('Q-Q Plot de Residuos ARIMA(1,0,1)')

plt.tight_layout()
plt.show()
```

Original markdown cell 26 from Series de Tiempo.ipynb

Interpretación de los Gráficos Q-Q

- **Línea de 45 grados (line='s'):** Esta línea diagonal representa la distribución normal teórica. Si los puntos de los residuos se agrupan estrechamente alrededor de esta línea, indica que los residuos siguen una distribución normal.
- **Desviaciones:**
 - Si los puntos se curvan en los extremos, sugiere colas más pesadas o más ligeras de lo esperado en una distribución normal.
 - Si los puntos se desvían de la línea en el centro, puede indicar asimetría en la distribución de los residuos.

En nuestro caso, esperamos que los residuos se acerquen a una distribución normal, ya que el término de ruido blanco (`epsilon_t`) que utilizamos para generar la `ar1_series` fue muestreado de una distribución normal. Por lo tanto, los gráficos Q-Q deberían mostrar puntos alineados muy cerca de la línea de 45 grados, confirmando la suposición de normalidad de los errores.

Original markdown cell 27 from Series de Tiempo.ipynb

Resumen de AIC y BIC para todos los Modelos ARIMA Evaluados

A continuación, se presenta un DataFrame que resume los valores de AIC y BIC para todas las combinaciones de órdenes ARIMA(p,0,q) que fueron evaluadas. Esto nos permite tener una visión general rápida de qué modelos tuvieron el mejor rendimiento según estos criterios.

Original code cell 28 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import pandas as pd

# Asegurarse de que 'results_ar1' y 'results_best_aic' estén disponibles
# Esto se maneja en la celda anterior donde se ajustan los modelos.

summary_df = pd.DataFrame({
    'Order (p,d,q)': model_orders,
    'AIC': aic_results,
    'BIC': bic_results
})

# Ordenar por AIC para ver los mejores modelos
print("\n--- Modelos ordenados por AIC ---")
display(summary_df.sort_values(by='AIC').reset_index(drop=True))

# Ordenar por BIC para ver los mejores modelos
print("\n--- Modelos ordenados por BIC ---")
display(summary_df.sort_values(by='BIC').reset_index(drop=True))
```

Original markdown cell 29 from Series de Tiempo.ipynb

Generación de Predicciones con el Modelo ARIMA(1,0,0)

Ahora que hemos seleccionado y validado nuestro modelo ARIMA(1,0,0) como el más adecuado, podemos usarlo para generar predicciones futuras de la serie temporal. Es importante recordar que este modelo se basa en la estructura AR(1) de la serie, por lo que sus predicciones intentarán seguir esa dinámica.

Original code cell 30 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import matplotlib.pyplot as plt

# Definir el número de pasos de tiempo para la predicción
forecast_steps = 20 # Por ejemplo, predecir los próximos 20 pasos de tiempo

# Generar las predicciones
# El método get_forecast() permite obtener el intervalo de confianza.
```

```

forecast_result = results_ar1.get_forecast(steps=forecast_steps)

# Obtener los valores predichos y sus intervalos de confianza
forecast = forecast_result.predicted_mean
conf_int = forecast_result.conf_int()

# Visualizar la serie original, el ajuste del modelo y las predicciones
plt.figure(figsize=(16, 8))
plt.plot(ar1_series, label='Serie Original', color='blue')
plt.plot(results_ar1.predict(start=0, end=len(ar1_series)-1), label='Ajuste del Modelo ARIMA(1',
plt.plot(range(len(ar1_series), len(ar1_series) + forecast_steps), forecast, label='Predicción',
plt.fill_between(range(len(ar1_series), len(ar1_series) + forecast_steps),
                 conf_int[:, 0], conf_int[:, 1], color='pink', alpha=0.3, label='Intervalo de (

plt.title('Serie Original, Ajuste del Modelo y Predicciones ARIMA(1,0,0)')
plt.xlabel('Paso de Tiempo')
plt.ylabel('Valor')
plt.legend()
plt.grid(True)
plt.show()

```

Original markdown cell 31 from Series de Tiempo.ipynb

1. Carga y Preparación de Datos desde URL

Primero, descargaremos un conjunto de datos de temperaturas mínimas diarias en Melbourne, Australia, disponible públicamente. Este dataset es ideal para Prophet ya que tiene una columna de fecha y una columna de valor, y probablemente muestre estacionalidad.

Original code cell 32 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

import pandas as pd
import matplotlib.pyplot as plt

# URL del dataset de temperaturas mínimas diarias
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/daily-min-temperatures.csv'
df = pd.read_csv(url, header=0)

# Mostrar las primeras filas y la información del DataFrame
print("Primeras 5 filas del dataset:")
display(df.head())
print("\nInformación del DataFrame:")
df.info()

# Prophet requiere que las columnas de fecha se llamen 'ds' y las de valor 'y'
# y que 'ds' sea de tipo datetime.

```

```

df['Date'] = pd.to_datetime(df['Date'])
df = df.rename(columns={'Date': 'ds', 'Temp': 'y'})

print("\nDataset preparado para Prophet (primeras 5 filas):")
display(df.head())

# Visualizar la serie temporal original
plt.figure(figsize=(12, 6))
plt.plot(df['ds'], df['y'])
plt.title('Temperaturas Mínimas Diarias en Melbourne')
plt.xlabel('Fecha')
plt.ylabel('Temperatura (°C)')
plt.grid(True)
plt.show()

```

Original markdown cell 33 from `Series de Tiempo.ipynb`

2. Instalación y Aplicación del Modelo Prophet

Ahora instalaremos la librería `prophet` (si aún no está instalada) y ajustaremos el modelo a nuestros datos. Prophet es robusto para manejar datos faltantes y cambios en la tendencia.

Original code cell 34 from `Series de Tiempo.ipynb` Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

# Instalar Prophet si es necesario
# !pip install prophet

from prophet import Prophet

# Crear una instancia del modelo Prophet
# Podemos añadir parámetros como la estacionalidad anual, semanal, etc.
model_prophet = Prophet(daily_seasonality=True)

# Ajustar el modelo a los datos
model_prophet.fit(df)

print("Modelo Prophet ajustado con éxito.")

```

Original markdown cell 35 from `Series de Tiempo.ipynb`

3. Generación de Predicciones con Prophet

Una vez que el modelo está ajustado, podemos generar un DataFrame con fechas futuras para realizar las predicciones. Vamos a predecir para el próximo año.

Original code cell 36 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
# Crear un DataFrame con fechas futuras para la predicción (ej. 365 días más)
future = model_prophet.make_future_dataframe(periods=365)

# Realizar las predicciones
forecast = model_prophet.predict(future)

print("Predicciones generadas (primeras 5 filas del forecast):")
display(forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].head())
```

Original markdown cell 37 from Series de Tiempo.ipynb

4. Visualización de Predicciones y Componentes del Modelo Prophet

Prophet facilita la visualización de las predicciones, incluyendo el histórico, el pronóstico y los intervalos de confianza. También podemos ver la descomposición de la serie en sus componentes (tendencia, estacionalidad).

Original code cell 38 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
# Gráfico de las predicciones
fig1 = model_prophet.plot(forecast)
plt.title('Predicción de Temperaturas Mínimas con Prophet')
plt.xlabel('Fecha')
plt.ylabel('Temperatura (°C)')
plt.grid(True)
plt.show()

# Gráfico de los componentes del modelo (tendencia, estacionalidad anual, semanal, diaria)
fig2 = model_prophet.plot_components(forecast)
plt.suptitle('Componentes del Modelo Prophet', y=1.02) # Ajustar título principal
plt.tight_layout(rect=[0, 0.03, 1, 0.98]) # Ajustar layout para evitar solapamiento
plt.show()
```

Original markdown cell 39 from Series de Tiempo.ipynb

5. Nota sobre AIC y BIC con Prophet

Como mencionamos al principio, **Prophet no utiliza AIC o BIC directamente** para la selección del modelo o para reportar la bondad de ajuste en la misma forma que lo hacen los modelos estadísticos tradicionales como ARIMA.

Prophet se basa en un enfoque de modelado de series de tiempo con componentes aditivos o multiplicativos (tendencia, estacionalidades y días festivos). La evaluación y selección de modelos en Prophet típicamente se realiza a través de:

- **Validación cruzada:** Para evaluar el rendimiento predictivo del modelo en datos no vistos.
- **Métricas de error:** Como RMSE (Error Cuadrático Medio), MAE (Error Absoluto Medio) o MAPE (Error Porcentual Absoluto Medio), que miden la precisión del pronóstico.
- **Análisis visual:** De las predicciones y sus componentes, como las gráficas que acabamos de generar.

Por lo tanto, en el contexto de Prophet, la pregunta sobre un "buen modelo" se responde más por su rendimiento predictivo y la coherencia de sus componentes con las expectativas del dominio, más que por criterios de información como AIC o BIC.

Original markdown cell 40 from Series de Tiempo.ipynb

Interpretación de la Gráfica de Predicción

La gráfica anterior muestra:

- **Serie Original (azul):** Los datos que hemos analizado y modelado.
- **Ajuste del Modelo (púrpura):** Cómo el modelo ARIMA(1,0,0) se ajusta a los datos históricos.
- **Predicción (rojo):** Los valores futuros estimados por el modelo para los próximos `forecast_steps` (20 en este caso) pasos de tiempo.
- **Intervalo de Confianza (rosa):** Una banda alrededor de la predicción que nos da una idea de la incertidumbre asociada a estas estimaciones. A medida que nos alejamos en el futuro, es normal que el intervalo de confianza se ensanche, reflejando el aumento de la incertidumbre.

Esta visualización es crucial para entender no solo las estimaciones puntuales del modelo, sino también la fiabilidad de dichas predicciones. Dado que la serie fue generada por un proceso AR(1) estacionario, esperamos que las predicciones tiendan a la media de la serie (que es cero en nuestro caso) a medida que el horizonte de predicción se extiende, y que la banda de confianza refleje esta convergencia y la incertidumbre inherente.

Original markdown cell 41 from Series de Tiempo.ipynb

Detalle Completo de los Modelos "Óptimos"

Para un análisis a máximo detalle, presentamos los resúmenes completos de los modelos seleccionados como óptimos por AIC y BIC. Estas tablas incluyen coeficientes de los parámetros, errores estándar, valores p, y otras estadísticas importantes para evaluar la significancia y el ajuste del modelo.

Original code cell 42 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
print("\n--- Resumen Detallado del Modelo ARIMA(1,0,0) (Mejor según BIC) ---")
display(results_ar1.summary())
```

Original code cell 43 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
print("\n--- Resumen Detallado del Modelo ARIMA(1,0,1) (Mejor según AIC) ---")
display(results_best_aic.summary())
```

Original markdown cell 44 from Series de Tiempo.ipynb

Interpretación de los Gráficos de Residuos

Al observar los gráficos de residuos, buscamos las siguientes características:

- **Ruido Blanco:** Los residuos deben estar distribuidos aleatoriamente alrededor de cero. Esto significa que el modelo ha capturado la mayor parte de la estructura sistemática en la serie temporal.
- **Sin Patrones:** No debería haber patrones visibles (tendencias, estacionalidad, agrupaciones de valores positivos/negativos). La presencia de patrones sugiere que el modelo no ha capturado completamente la información en los datos.
- **Varianza Constante:** La dispersión de los residuos debe ser más o menos constante a lo largo del tiempo (homocedasticidad).
- **Autocorrelación Mínima:** Idealmente, no debería haber autocorrelación significativa en los residuos, lo que se puede verificar con gráficas ACF/PACF de los propios residuos (no lo haremos aquí para simplificar, pero es un paso importante en un análisis real).

Dado que nuestra `ar1_series` fue generada por un proceso AR(1) puro, esperamos que los residuos del modelo ARIMA(1,0,0) se acerquen mucho al ruido blanco. Si el modelo ARIMA(1,0,1) muestra residuos muy similares, esto reforzaría la idea de que el término MA adicional no aportó mucha información nueva, confirmando la preferencia del BIC por el modelo más simple.

Original code cell 45 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import warnings
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore") # Ignorar warnings de convergencia

# Definir el rango de órdenes p y q a probar
p_values = range(0, 4) # p de 0 a 3
q_values = range(0, 4) # q de 0 a 3
d_value = 0 # Mantener d=0 ya que la serie es estacionaria

aic_results = []
bic_results = []
model_orders = []

for p in p_values:
    for q in q_values:
        order = (p, d_value, q)
```



```

if p == 0 and q == 0:
    continue # Evitar el modelo (0,0,0) que no tiene parámetros
try:
    model = ARIMA(ar1_series, order=order)
    model_fit = model.fit()
    aic_results.append(model_fit.aic)
    bic_results.append(model_fit.bic)
    model_orders.append(str(order))
    print(f'ARIMA{order} - AIC: {model_fit.aic:.2f}, BIC: {model_fit.bic:.2f}')
except:
    # print(f'No se pudo ajustar ARIMA{order}')
    pass # Algunos modelos pueden no converger

# Encontrar el mejor modelo basado en AIC y BIC
best_aic_index = aic_results.index(min(aic_results))
best_bic_index = bic_results.index(min(bic_results))

print(f"\nMejor modelo según AIC: {model_orders[best_aic_index]} con AIC = {aic_results[best_aic_index]}")
print(f"Mejor modelo según BIC: {model_orders[best_bic_index]} con BIC = {bic_results[best_bic_index]}")

```

Original markdown cell 46 from Series de Tiempo.ipynb

Gráficas de AIC y BIC por Orden de Modelo

Visualicemos los valores de AIC y BIC para cada modelo ajustado. Esto nos permitirá ver claramente qué órdenes (p, q) minimizan estos criterios.

Original code cell 47 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

plt.figure(figsize=(16, 8))

plt.plot(model_orders, aic_results, marker='o', linestyle='-', color='blue', label='AIC')
plt.plot(model_orders, bic_results, marker='x', linestyle='--', color='red', label='BIC')

plt.title('Comparación de AIC y BIC para Diferentes Órdenes ARIMA(p,0,q)')
plt.xlabel('Orden del Modelo (p,0,q)')
plt.ylabel('Valor del Criterio')
plt.xticks(rotation=45, ha='right')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Original markdown cell 48 from Series de Tiempo.ipynb

Identificación de P y Q a partir de las Gráficas ACF y PACF

Las gráficas de Autocorrelación (ACF) y Autocorrelación Parcial (PACF) son herramientas visuales fundamentales para determinar los órdenes p (Autorregresivo) y q (Media Móvil) de un modelo ARIMA.

Para nuestra serie `ar1_series`, que fue generada por un proceso AR(1) ($X_t = c + \phi X_{t-1} + \epsilon_t$), esperábamos un comportamiento específico:

1. Determinación de P (Orden del Componente Autorregresivo):

- Observamos la **gráfica de PACF (Autocorrelación Parcial)**. Para un proceso AR(p), la PACF se "corta" o cae abruptamente a cero (o dentro de las bandas de confianza) después del rezago p .
- En nuestra gráfica de PACF, vemos un **pico significativo en el rezago 1**, y luego las autocorrelaciones parciales caen rápidamente a valores no significativos (dentro de las bandas azules de confianza).
- Esto indica claramente que **$P = 1$** , lo cual es consistente con nuestra serie AR(1).

2. Determinación de Q (Orden del Componente de Media Móvil):

- Observamos la **gráfica de ACF (Autocorrelación)**. Para un proceso MA(q), la ACF se "corta" o cae abruptamente a cero (o dentro de las bandas de confianza) después del rezago q .
- Sin embargo, para un proceso AR(p) puro (como nuestra `ar1_series`), la ACF no se "corta" abruptamente; en su lugar, **decae gradualmente**, a menudo de forma exponencial o sinusoidal, a medida que los rezagos aumentan.
- En nuestra gráfica de ACF, observamos un decaimiento gradual, lo que sugiere que **$Q = 0$** si el proceso es un AR puro. Si hubiera un "corte" abrupto en un rezago específico, ese sería nuestro q .

Original markdown cell 49 from `Series de Tiempo.ipynb`

Interpretación de las Gráficas de AIC y BIC

Al observar estas gráficas, el modelo con el punto más bajo en la línea de AIC o BIC es el que se considera óptimo según ese criterio. Es común que BIC favorezca modelos más simples que AIC, especialmente con conjuntos de datos grandes, debido a su penalización más fuerte por la complejidad del modelo.

En el caso de nuestra `ar1_series` que fue generada por un proceso AR(1), esperaríamos que el modelo ARIMA(1,0,0) (que es equivalente a un AR(1)) tuviera los valores más bajos de AIC y BIC, o al menos que BIC lo favoreciera por su simplicidad.

Original markdown cell 50 from `Series de Tiempo.ipynb`

Visualización del Ajuste del Mejor Modelo ARIMA

Basándonos en el análisis de AIC y BIC, especialmente considerando que la serie fue generada por un proceso AR(1), el modelo ARIMA(1,0,0) es el más parsimonioso y adecuado según el BIC. Vamos a graficar su ajuste sobre la serie original para ver visualmente cómo se comporta.

Original code cell 51 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA

# El modelo AR(1) ya fue ajustado y está disponible como results_ar1, que es ARIMA(1,0,0)
# Si quisieras usar el mejor AIC (1,0,1) o cualquier otro, lo ajustarías aquí:
# model_best_aic = ARIMA(ar1_series, order=(1, 0, 1))
# results_best_aic = model_best_aic.fit()

plt.figure(figsize=(14, 7))
plt.plot(ar1_series, label='Serie Original', color='blue')
plt.plot(results_ar1.predict(start=0, end=len(ar1_series)-1), label='Ajuste ARIMA(1,0,0)', color='red')
plt.title('Serie Original vs. Ajuste del Modelo ARIMA(1,0,0)')
plt.xlabel('Paso de Tiempo')
plt.ylabel('Valor')
plt.legend()
plt.grid(True)
plt.show()
```

Original markdown cell 52 from Series de Tiempo.ipynb

Interpretación de la Gráfica del Modelo ARIMA

Esta gráfica nos muestra cómo el modelo ARIMA(1,0,0) (que es equivalente a un modelo AR(1)) se ajusta a la serie `ar1_series` original. Deberías observar que la línea de ajuste sigue de cerca los patrones de la serie original, lo que indica un buen rendimiento del modelo para capturar la dinámica de los datos. Esta visualización complementa la información proporcionada por los criterios AIC y BIC, confirmando que el modelo seleccionado es una representación adecuada de la serie temporal.

Original markdown cell 53 from Series de Tiempo.ipynb

Conclusión General del Análisis de Modelos AR y ARIMA

En este análisis hemos explorado los fundamentos de la modelización de series temporales, centrándonos en los modelos Autorregresivos (AR) y ARIMA, y cómo se utilizan los Criterios de Información de Akaike (AIC) y Bayesiano (BIC) para la selección de modelos.

1. Generación y Visualización de la Serie Temporal:

Simulamos una serie `ar1_series` a partir de un proceso AR(1), lo que nos dio un punto de referencia claro para evaluar el rendimiento de nuestros modelos.

2. Comparación de Modelos AR con AIC y BIC:

Ajustamos un modelo AR(1) y un AR(2) a la serie generada. Observamos que:

- El **AIC** favoreció ligeramente al modelo AR(2), lo que sugiere que pudo haber capturado alguna pequeña variación adicional.

- El **BIC**, con su penalización más estricta por complejidad, seleccionó el modelo AR(1) como el mejor. Esto es coherente con la naturaleza de la serie generada y subraya la preferencia del BIC por modelos más parsimoniosos cuando la mejora de ajuste no justifica la complejidad adicional.

Las gráficas de ajuste visual confirmaron que ambos modelos replicaron bastante bien la serie original, mientras que el análisis de residuos mostró qué tan cerca cada modelo se acercó a un comportamiento de "ruido blanco".

3. Modelos ARIMA y Funciones ACF/PACF:

Repasamos la teoría detrás de los modelos ARIMA y la importancia de las Funciones de Autocorrelación (ACF) y Autocorrelación Parcial (PACF) para determinar los órdenes p (AR) y q (MA):

- En la **gráfica PACF** de nuestra `ar1_series`, el pico significativo en el **rezago 1** y la caída abrupta posterior claramente señalaron un orden $p=1$.
- En la **gráfica ACF**, el decaimiento gradual (sin un corte abrupto) es consistente con un modelo AR(1) puro, lo que sugiere un orden $q=0$.

4. Ajuste de Modelos ARIMA e Interpretación Gráfica de AIC/BIC:

Iteramos sobre diferentes combinaciones de órdenes (p , 0 , q) para los modelos ARIMA y calculamos sus valores de AIC y BIC. La gráfica de estos criterios nos permitió visualizar que el modelo **ARIMA(1,0,0)** (equivalente a un AR(1)) fue el más consistente con los resultados del BIC, y tuvo un AIC competitivo, lo que nuevamente reforzó que el proceso AR(1) era el subyacente.

5. Visualización del Ajuste del Mejor Modelo ARIMA:

Finalmente, graficamos el ajuste del modelo ARIMA(1,0,0) sobre la serie original. Esta visualización demostró que el modelo seleccionado es una representación robusta y precisa de la dinámica de nuestra serie temporal simulada.

En resumen, este ejercicio ha ilustrado cómo el AIC y el BIC, junto con las herramientas visuales como las gráficas de ACF, PACF y los ajustes de modelos, son fundamentales para la identificación y selección del modelo más apropiado en el análisis de series temporales.

Original code cell 54 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import numpy as np
import matplotlib.pyplot as plt

# Calculate ZT using the formula  $ZT = XT - 0.5 * XT-1$ 
# Note: XT is represented by ar1_series

# Initialize ZT series, it will have n-1 elements if starting from t=1
zt_series = np.zeros(n)

# ZT[0] cannot be calculated as XT[-1] is not defined in this context.
# Let's assume ZT starts from the second element, corresponding to XT[1].
```

```

# Alternatively, we can define ZT[0] based on XT[0] if XT[-1] is assumed 0 or use the first ca
# For simplicity, let's calculate from t=1 to n-1
# In the context of the AR(1) process  $X_t = c + \phi * X_{t-1} + \epsilon_t$ ,
# if  $\phi = 0.5$ , then  $\epsilon_t = X_t - 0.5 * X_{t-1}$ . So ZT is essentially the white noise term.

# Let's calculate ZT as epsilon_t directly from the formula given.
# We already have white_noise which is epsilon_t.
zt_series = white_noise

# Plot the simulated ZT series (which is the white noise / innovations)
plt.figure(figsize=(12, 6))
plt.plot(zt_series)
plt.title('Simulated ZT Series (Innovations)')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.grid(True)
plt.show()

```

Original code cell 55 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

ventas = pd.read_csv('california_housing_train.csv')
ventas.head(101)

```

Original code cell 56 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

pltr = yf.download("BK", start="1900-01-01")
pltr

```

Original code cell 57 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

pltr_price=pltr['Close']
pltr_price

```

Original code cell 58 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

import plotly.express as px

fig = px.line(pltr_price, x=pltr_price.index, y='PLTR', title='PLTR Close Price Over Time')
fig.show()

```

Original code cell 59 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

Original markdown cell 60 from Series de Tiempo.ipynb

Modelo SARIMA (Seasonal ARIMA)

El modelo SARIMA (Seasonal AutoRegressive Integrated Moving Average) extiende el modelo ARIMA para incluir componentes estacionales. Se especifica como $\text{SARIMA}(p, d, q)(P, D, Q)s$, donde:

- (p, d, q) son las órdenes no estacionales (AR, I, MA).
- (P, D, Q) son las órdenes estacionales (AR, I, MA).
- s es la longitud del período estacional (por ejemplo, 7 para datos diarios con estacionalidad semanal, 12 para datos mensuales con estacionalidad anual).

Para este ejercicio, ajustaremos un modelo $\text{SARIMA}(0,1,1)(0,1,1)$ con un período estacional $s=7$ a la serie de temperaturas mínimas diarias de Melbourne, lo que implica:

- No hay componente AR no estacional ($p=0$).
- Una diferenciación no estacional ($d=1$), para manejar tendencias.
- Un componente MA no estacional ($q=1$).
- No hay componente AR estacional ($P=0$).
- Una diferenciación estacional ($D=1$), para manejar estacionalidad.
- Un componente MA estacional ($Q=1$).
- Un período estacional de 7 días ($s=7$), asumiendo una estacionalidad semanal en las temperaturas diarias.

Original code cell 61 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import statsmodels.api as sm
import matplotlib.pyplot as plt
import pandas as pd

# Asegurarse de que el DataFrame 'df' de temperaturas esté disponible y con el formato correcto
# Ya fue preparado en la celda 0ee811c0
# df['ds'] = pd.to_datetime(df['ds'])
# df = df.set_index('ds')

# Definir los órdenes del modelo SARIMA
order = (0, 1, 1)          # (p, d, q) no estacional
seasonal_order = (0, 1, 1, 7) # (P, D, Q, s) estacional, con s=7 para estacionalidad semanal

print(f"Ajustando modelo SARIMA{order}{seasonal_order}...")
```

```

# Ajustar el modelo SARIMA
try:
    # Usamos la serie 'y' que contiene las temperaturas
    model_sarima = sm.tsa.SARIMAX(df['y'], order=order, seasonal_order=seasonal_order, enforce_stationarity=False, enforce_invertibility=False)
    results_sarima = model_sarima.fit(dispatch=False) # disp=False para evitar la salida de conversión de unidades
    print("Modelo SARIMA ajustado con éxito.")
    print(results_sarima.summary())
except Exception as e:
    print(f"Error al ajustar el modelo SARIMA: {e}")

```

Original code cell 62 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

import matplotlib.pyplot as plt

# Generar predicciones para los datos históricos
# start y end se refieren a las posiciones en el índice, no a las fechas.
# La serie original df['y'] tiene un índice de 0 a n-1, si no se seteo el índice de fecha.
# Asumiendo que df['y'] es una serie con un índice DatetimeIndex

# Obtener las fechas del DataFrame original para el plot
original_dates = df['ds']

# Predicciones in-sample (ajuste del modelo a los datos históricos)
# Usamos el índice de `df` que ya tiene la columna 'ds' como datetime
start_index_fit = 0
end_index_fit = len(df['y']) - 1
# Las predicciones se alinearán con el índice de tiempo de la serie original.
predictions_sarima = results_sarima.predict(start=start_index_fit, end=end_index_fit, dynamic=False)

# Generar pronósticos para el futuro (ej. los próximos 30 días)
forecast_steps = 30
forecast_sarima_result = results_sarima.get_forecast(steps=forecast_steps)
forecast_sarima = forecast_sarima_result.predicted_mean
conf_int_sarima = forecast_sarima_result.conf_int()

# Crear un índice de fechas para el pronóstico
last_date_original = original_dates.max()
forecast_dates = pd.date_range(start=last_date_original + pd.Timedelta(days=1), periods=forecast_steps)

plt.figure(figsize=(18, 8))
plt.plot(original_dates, df['y'], label='Serie Original', color='blue')
plt.plot(original_dates, predictions_sarima, label='Ajuste SARIMA', color='green', linestyle='--')
plt.plot(forecast_dates, forecast_sarima, label=f'Pronóstico SARIMA ({forecast_steps} días)', color='red', linestyle='--')
plt.fill_between(forecast_dates,
                 conf_int_sarima.iloc[:, 0],
                 conf_int_sarima.iloc[:, 1],
                 color='red', alpha=0.2)

```

```

        color='pink', alpha=0.3, label='Intervalo de Confianza (95%)')

plt.title(f'Serie Original, Ajuste y Pronóstico con SARIMA{order}{seasonal_order}')
plt.xlabel('Fecha')
plt.ylabel('Temperatura (°C)')
plt.legend()
plt.grid(True)
plt.show()

print("\n--- Pronósticos Futuros del Modelo SARIMA ---")
# Combine forecast dates with forecast values and confidence intervals for display
forecast_df = pd.DataFrame({
    'Fecha': forecast_dates,
    'Pronóstico': forecast_sarima.values,
    'Límite Inferior (95%)': conf_int_sarima.iloc[:, 0].values,
    'Límite Superior (95%)': conf_int_sarima.iloc[:, 1].values
})
display(forecast_df)

```

Original markdown cell 63 from Series de Tiempo.ipynb

Búsqueda del Modelo SARIMA Óptimo usando AIC y BIC

Para encontrar el modelo SARIMA más adecuado para nuestras temperaturas, vamos a realizar una búsqueda exhaustiva (grid search) a través de diferentes combinaciones de órdenes no estacionales (p , d , q) y estacionales (P , D , Q , s). Evaluaremos cada modelo utilizando los criterios AIC (Akaike Information Criterion) y BIC (Bayesian Information Criterion). El modelo con los valores más bajos de AIC y/o BIC será considerado el 'óptimo', ya que estos criterios penalizan la complejidad del modelo al mismo tiempo que evalúan su ajuste a los datos.

Mantendremos $d=1$ y $D=1$ para la diferenciación no estacional y estacional respectivamente, ya que la serie de temperaturas mínimas diarias suele presentar tendencia y estacionalidad. El período estacional s se fijará en 7 (semanal).

Original code cell 64 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

import warnings
import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore") # Ignorar warnings de convergencia

# Definir los rangos de órdenes p, q, P, Q a probar
p_values = range(0, 2) # p de 0 a 1
q_values = range(0, 2) # q de 0 a 1
P_values = range(0, 2) # P de 0 a 1

```



```

Q_values = range(0, 2) # Q de 0 a 1
d_value = 1           # Diferenciación no estacional (fijo)
D_value = 1           # Diferenciación estacional (fijo)
s_value = 7           # Período estacional (fijo para semanal)

aic_results_sarima = []
bic_results_sarima = []
model_orders_sarima = []

# Asegurarse de que el DataFrame 'df' de temperaturas esté disponible y con el formato correcto
# Ya fue preparado en la celda 0ee811c0
# Establecer la columna 'ds' como índice si no lo está para SARIMAX
# sarimax_df = df.set_index('ds')

print("Iniciando búsqueda de modelos SARIMA...")

for p in p_values:
    for q in q_values:
        for P in P_values:
            for Q in Q_values:
                order = (p, d_value, q)
                seasonal_order = (P, D_value, Q, s_value)

                if p == 0 and q == 0 and P == 0 and Q == 0: # Evitar el modelo (0,1,0)(0,1,0)7
                    continue

                try:
                    model = SARIMAX(df['y'], order=order, seasonal_order=seasonal_order,
                                    enforce_stationarity=False, enforce_invertibility=False)
                    model_fit = model.fit(dispatch=False) # dispatch=False para evitar la salida de console
                    aic_results_sarima.append(model_fit.aic)
                    bic_results_sarima.append(model_fit.bic)
                    model_orders_sarima.append(f'SARIMA{order}{seasonal_order}')
                    print(f'SARIMA{order}{seasonal_order} - AIC: {model_fit.aic:.2f}, BIC: {model_fit.bic:.2f}')
                except Exception as e:
                    # print(f'No se pudo ajustar SARIMA{order}{seasonal_order}: {e}')
                    pass # Algunos modelos pueden no converger o dar errores

# Encontrar el mejor modelo basado en AIC y BIC
if aic_results_sarima and bic_results_sarima:
    best_aic_index_sarima = aic_results_sarima.index(min(aic_results_sarima))
    best_bic_index_sarima = bic_results_sarima.index(min(bic_results_sarima))

    print(f"\nMejor modelo según AIC: {model_orders_sarima[best_aic_index_sarima]} con AIC = {aic_results_sarima[best_aic_index_sarima]:.2f}")
    print(f"Mejor modelo según BIC: {model_orders_sarima[best_bic_index_sarima]} con BIC = {bic_results_sarima[best_bic_index_sarima]:.2f}")
else:
    print("No se pudieron ajustar modelos SARIMA válidos con los parámetros seleccionados.")

```

```

# Crear un DataFrame para un resumen de los resultados
summary_sarima_df = pd.DataFrame({
    'Order': model_orders_sarima,
    'AIC': aic_results_sarima,
    'BIC': bic_results_sarima
})

print("\n--- Resumen de Modelos SARIMA Ordenados por AIC ---")
display(summary_sarima_df.sort_values(by='AIC').reset_index(drop=True))

print("\n--- Resumen de Modelos SARIMA Ordenados por BIC ---")
display(summary_sarima_df.sort_values(by='BIC').reset_index(drop=True))

```

Original code cell 65 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

plt.figure(figsize=(16, 8))

plt.plot(model_orders_sarima, aic_results_sarima, marker='o', linestyle='-', color='blue', label='AIC')
plt.plot(model_orders_sarima, bic_results_sarima, marker='x', linestyle='--', color='red', label='BIC')

plt.title('Comparación de AIC y BIC para Diferentes Órdenes SARIMA')
plt.xlabel('Orden del Modelo (p,d,q)(P,D,Q,s)')
plt.ylabel('Valor del Criterio')
plt.xticks(rotation=90, ha='right')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Original markdown cell 66 from Series de Tiempo.ipynb

Visualización del Ajuste y Pronóstico del Modelo SARIMA Óptimo

Una vez identificados los modelos SARIMA óptimos según AIC y BIC, vamos a visualizar el ajuste del mejor modelo (considerando el equilibrio entre ajuste y parsimonia) a los datos históricos y sus pronósticos futuros. En muchos casos, el modelo con el BIC más bajo es preferible por su mayor penalización a la complejidad.

Original code cell 67 from Series de Tiempo.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```

import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
import re # Para extraer los órdenes del string

# Obtener el mejor modelo según BIC (que tiende a ser más parsimonioso)

```

```

best_bic_order_str = model_orders_sarima[best_bic_index_sarima]

# Extraer los órdenes (p,d,q) y (P,D,Q,s) del string
match = re.match(r'SARIMA\((\d),\s*(\d),\s*(\d)\)\((\d),\s*(\d),\s*(\d),\s*(\d)\)', best_bic_order_str)

if match:
    p_opt, d_opt, q_opt, P_opt, D_opt, Q_opt, s_opt = map(int, match.groups())
    optimal_order = (p_opt, d_opt, q_opt)
    optimal_seasonal_order = (P_opt, D_opt, Q_opt, s_opt)
else:
    print(f"No se pudieron extraer los órdenes del string: {best_bic_order_str}. Usando valores por defecto")
    optimal_order = (0, 1, 1)
    optimal_seasonal_order = (0, 1, 1, 7)

print(f"Ajustando el modelo SARIMA óptimo: SARIMA{optimal_order}{optimal_seasonal_order}")

# Ajustar el modelo SARIMA óptimo
model_optimal_sarima = SARIMAX(df['y'], order=optimal_order, seasonal_order=optimal_seasonal_order,
                                enforce_stationarity=False, enforce_invertibility=False)
results_optimal_sarima = model_optimal_sarima.fit(dispatch=False)

# Obtener las fechas del DataFrame original para el plot
original_dates = df['ds']

# Predicciones in-sample (ajuste del modelo a los datos históricos)
start_index_fit = 0
end_index_fit = len(df['y']) - 1
predictions_optimal_sarima = results_optimal_sarima.predict(start=start_index_fit, end=end_index_fit)

# Generar pronósticos para el futuro (ej. los próximos 30 días)
forecast_steps_optimal = 30
forecast_optimal_sarima_result = results_optimal_sarima.get_forecast(steps=forecast_steps_optimal)
forecast_optimal_sarima = forecast_optimal_sarima_result.predicted_mean
conf_int_optimal_sarima = forecast_optimal_sarima_result.conf_int()

# Crear un índice de fechas para el pronóstico
last_date_original = original_dates.max()
forecast_dates_optimal = pd.date_range(start=last_date_original + pd.Timedelta(days=1), periods=forecast_steps_optimal)

plt.figure(figsize=(18, 8))
plt.plot(original_dates, df['y'], label='Serie Original', color='blue')
plt.plot(original_dates, predictions_optimal_sarima, label=f'Ajuste SARIMA{optimal_order}{optimal_seasonal_order}', color='red')
plt.plot(forecast_dates_optimal, forecast_optimal_sarima, label=f'Pronóstico SARIMA{optimal_order}{optimal_seasonal_order}', color='red')
plt.fill_between(forecast_dates_optimal,
                 conf_int_optimal_sarima.iloc[:, 0],
                 conf_int_optimal_sarima.iloc[:, 1],
                 color='pink', alpha=0.3, label='Intervalo de Confianza (95%)')

```

```

plt.title(f'Serie Original, Ajuste y Pronóstico con SARIMA{optimal_order}-{optimal_seasonal_order}')
plt.xlabel('Fecha')
plt.ylabel('Temperatura (°C)')
plt.legend()
plt.grid(True)
plt.show()

print("\n--- Pronósticos Futuros del Modelo SARIMA Óptimo ---")
# Combine forecast dates with forecast values and confidence intervals for display
forecast_optimal_df = pd.DataFrame({
    'Fecha': forecast_dates_optimal,
    'Pronóstico': forecast_optimal_sarima.values,
    'Límite Inferior (95%)': conf_int_optimal_sarima.iloc[:, 0].values,
    'Límite Superior (95%)': conf_int_optimal_sarima.iloc[:, 1].values
})
display(forecast_optimal_df)

```

Calculo Estocastico: Probability sample spaces and Bayes' theorem

Objective. Illustrate basic probability concepts: sample spaces, sets, Cartesian products, dice outcomes, and Bayes' theorem.

Inputs/parameters. Finite set ($\Omega = \{a, s\}$); two-fold Cartesian product; two dice with values 1 through 6.

Method. The code creates sets, checks type/cardinality, forms ordered pairs with `itertools.product`, creates a 36-element dice sample space, and prints events corresponding to each possible sum.

Key formulas. ($|\Omega| = 2$), ($|\Omega^2| = 4$), and for two dice ($|\Omega| = 36$). Bayes' theorem: $P(A \mid B) = P(B \mid A)P(A)/P(B)$.

Code explanation. The first source cell contains a Markdown/LaTeX statement inside a code cell. That is execution-blocking as Python, so in the consolidated notebook it is preserved as a raw source note and explicitly flagged.

Output interpretation. Saved outputs show `set`, cardinality 2, one element access from the set converted to list, four ordered pairs, 36 dice outcomes, and event subsets for sums 1 through 12. The event for sum 1 is empty, as expected because two dice cannot sum to 1.

Assumptions/limitations. Set order is not guaranteed; `list(omega)[1]` can change across runs. The dice loop runs through 1 to 12 even though feasible sums are 2 through 12.

Original cell 0 from Calculo Estocastico.ipynb preserved as raw text This source cell was marked as code in the original notebook but contains Markdown/LaTeX prose, so it would raise a Python syntax error. It is preserved below as raw source rather than silently rewritten as executable Python.

Bayes' theorem (for two events A and B , with $P(B) \neq 0$) is:

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

Original code cell 1 from Calculo Estocastico.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import scipy.stats as stats
import itertools as itool
import sklearn as skl
```

Original code cell 2 from Calculo Estocastico.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
from itertools import product
```

Original code cell 3 from Calculo Estocastico.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
#Consideren al siguiente conjunto
omega={"a","s"}
```

Original code cell 4 from Calculo Estocastico.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
type(omega)
```

Original code cell 5 from Calculo Estocastico.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
print(len(omega))
```

Original code cell 6 from Calculo Estocastico.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
list(omega)[1]
```

Original code cell 7 from Calculo Estocastico.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
omega_dos = set(product (omega,repeat=2))
print (omega_dos)
```

Original code cell 8 from Calculo Estocastico.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
# Espacio muestral de dos dados (1-6 cada uno)
sample_space = [(d1, d2) for d1 in range(1, 7) for d2 in range(1, 7)]

print(len(sample_space)) # Debería imprimir 36
print(sample_space)      # Lista de pares (dado1, dado2)
```

Original code cell 9 from Calculo Estocastico.ipynb Source code copied exactly from the original notebook. Outputs are not fabricated; observed saved outputs are interpreted in the documentation above.

```
i=1
for i in range (1,13):
    s={o for o in sample_space if (o[0]+ o[1])==i}
    print(s)
    i=i+1
#
```

Results Interpretation

The observed outputs should be read as examples from the saved notebook state, not as permanent market facts. Any code using `yfinance`, Yahoo option chains, or external datasets is time-sensitive. If rerun on a later date, prices, option chains, maturities, implied volatilities, and fitted parameters may change.

The finance notebooks progressively distinguish three layers of modelling:

1. **Descriptive historical statistics.** Log returns, rolling volatility, and GARCH describe what happened in the historical sample.
2. **Model-based simulation.** GBM projects future paths under either historical drift (μ) or risk-neutral drift (r), depending on whether the goal is scenario simulation or pricing.
3. **Derivative valuation.** European, American, and Parisian options require discounted payoff logic. The difference among them is the payoff/exercise/barrier rule, not merely the simulation engine.

The time-series notebooks reinforce that model selection is not only visual. ACF/PACF guide candidate orders, but AIC/BIC quantify fit adjusted for complexity. When AIC and BIC disagree, the documentation should state the criterion used and why. In the saved ARIMA grid, AIC preferred a more complex model while BIC preferred the parsimonious ARIMA(1,0,0), which is consistent with BIC's stronger penalty.

The documented limitations matter. A notebook that works in a live, stateful classroom session may not run from top to bottom in a clean kernel. The consolidated version therefore preserves the original code and labels state dependencies instead of hiding them.

Assumptions and Limitations

- External data calls are not deterministic. `yfinance`, Yahoo option chains, GitHub-hosted CSVs, and package installations can change or fail.
- Saved outputs may be stale relative to current market data.
- Historical volatility is not implied volatility and should not be treated as a market forecast.
- GBM assumes lognormal prices, constant volatility, continuous compounding, and normally distributed shocks.
- Monte Carlo results depend on random seeds, path count, time discretization, and payoff definition.
- Longstaff-Schwartz results depend on basis functions, regression stability, and the simulated path distribution.
- Parisian barrier pricing is discretely monitored in the notebook; continuous monitoring is not implemented.
- GARCH conditional volatility is not a directional stock-price forecast.
- Prophet is not evaluated with AIC/BIC in the same way as likelihood-based ARIMA/SARIMA models.
- Some source notebooks are not cleanly executable from a fresh kernel because they rely on earlier hidden state or local files.

Appendix: Source Notebook Mapping

Source notebook	Original code cells reviewed	Original markdown cells reviewed	Integration status
Laboratorio 1.ipynb	1	0	Integrated and doc
Laboratorio 2.ipynb	1	0	Integrated and doc
Laboratorio 3.ipynb	1	0	Integrated and doc
Laboratorio 4.ipynb	1	0	Integrated and doc
Proyecto 4.ipynb	1	0	Integrated and doc
Untitled5.ipynb	1	0	Integrated and doc
Untitled3.ipynb	5	1	Integrated and doc
Series de Tiempo.ipynb	33	35	Integrated and doc
Calculo Estocastico.ipynb	10	0	Integrated and doc

Pre-final Verification Checklist

- All nine attached notebooks were reviewed.
- The output is one consolidated document, not separate documents per notebook.
- Every major code section is documented with objective, inputs, method, key formulas, explanation, output interpretation, and assumptions/limitations.
- Original code logic is preserved and copied without refactoring, optimizing, or silently correcting.
- Execution-blocking or fragile code is flagged explicitly.
- The `.ipynb`, `.md`, and `.tex` deliverables are generated from the same consolidated structure.
- LaTeX uses sections, equations, tables, and verbatim code formatting.
- Markdown is notebook-ready and readable.